SENG202 Project Document

Iteration 3

Project: Route Beer

Team 5

Classification: In-Confidence

Theo Baker, Alistair Clarke, Neela van Gerwen, Conrad Wilson

Table of Contents

Changelog *3*

Business and System Context *6*

Background **6**

Product Overview **7**

Current Market **8**

Competitors: 8

Market Gap 10

System Context **11**

Stakeholders *14*

Use Cases *17*

App Entry **25**

Quality Requirements *26*

Design decisions **30**

Verification **31**

Key Driver Analysis **32**

Acceptance Tests *34*

GUI Prototypes *39*

Final GUI Prototype ..... Error! Bookmark not defined.

Detailed UML Class Diagram (1-2 pages) *46*

Initial UML Class Diagram ..... Error! Bookmark not defined.

Testing (1 page) *47*

Testing process **55**

Discussion of Test Results **57**

Current Product Version (½-1 page) *60*

Deployment Model *62*

Risk Assessment *64*

Project Plan ***68***

References ..... *Error! Bookmark not defined.*

Appendix ***69***

Changelog

Table 1: Changelog of documentation with task, author and reviewer

Section	Changes/Responsibilities	Changed by	Reviewed by
Background	Started research into fields of interest related to Alcohol consumption and New Zealand History	Theo	Conrad
Product Overview	Noted down key concepts discussed during meetings and what functional requirements they meet	Theo	Neela
Current Market	Added basic intro summing key points. To be expanded on	Conrad	Alistair
Shareholders	Listed shared holders with ratings and expanded on descriptions	Alistar	Conrad
Current Market	Expanded on main introduction, Created three key weaknesses most competitors face. Strengths and weaknesses of each app type.	Conrad	Alistair
Quality Requirements	Created an outline of important requirements to be expanded on earlier	Theo	Conrad
Product Overview	Rewrote key components of the after final deliberation	Theo	Conrad
Stakeholders	Finished the stakeholder table	Alistair	Conrad
System context	First iteration of system context diagram and explanation	Neela	Conrad
Use cases	Defined all use cases and created tables for necessary use cases	Neela	Conrad
Risk Table	Risk assessment table with actions	Alistair	Theo
System context	Updated system context diagram	Neela	Conrad
Quality Requirements	Finalised quality requirement's to be in line with shareholders	Conrad	Alistair
Class diagram	Updated class diagram and wrote description	Neela	Alistair
Risk assessment	Done the risk assessment write up	Alistair	Theo
Acceptance test	The first 2/3 of the acceptance tests	Alistair	Conrad
Acceptance test	Rest of the draft acceptance tests	Conrad	Neela

Use cases	Refined use cases and reconstructed use case diagrams	Neela	Theo
Acceptance test	Refining the acceptance test to better align with the formatting and our use cases.	Alistair	Alistair Theo
Risk Assessment	Refactored the risk assessment after getting feedback from the tutors	Alistair	Theo
Key Driver Analysis	Completed from where Conrad has begun	Theo	Alistair
Risk Assessment	Reviewed	Theo	Neela
Project Plan	Inserted completed Gantt Chart	Theo	Alistair
UML Class Diagram Write up	Refactored the diagram and completed the write up for it	Neela	Alistair
DELIVERABLE 2			
Stakeholders	Integrated tourists and Christchurch residents into (non) abstinent end users	Alistair	Conrad
Adjusted system context diagram	Changed actor names	Neela	Alistair
Stakeholders	Updated stakeholder concerns to be more specific	Alistair	Theo
Current Market	Update the market gap/ our selling point	Alistair	Neela
System Context	Rejig some of the clarification around terminology of members/guests	Alistair	Theo
Use cases	Updated the use cases by making changes the teaching team suggested and moving them around	Alistair	Neela
Use case diagrams	Updated the three use case diagrams	Neela	Alistair
Project Plan	Updated project timeline	Alistair	Neela
Project Plan	Reworking the project plan to get rid of the fodder and add the project update	Alistair	Conrad
Background	Adding references to back up statistics	Alistair	Theo
Quality Requirements	Reanalysed quality requirements and determined what was required	Theo	Neela
Current Market	Adding the description and reasoning about the competitors and what they do well / poorly	Alistair	Conrad
Stakeholder	Refactoring the stakeholder write-up to cater for the changes made	Alistair	Theo
Detailed UML Class Diagram	Created the UML diagrams to represent our application	Theo	Neela
Use Case	Re writing UC_8 to cater for the more important use case in signing in	Alistair	Theo

Acceptance Testing	Making all the initial changes suggested by the teaching team and deleting obsolete tests	Alistair	Theo
Write about project testing	Wrote about the testing process of our project	Conrad	Neela
Acceptance testing	Adding four new tests. Refactoring just about every AT to align with the cucumber format better.	Alistair	Neela
Detailed UML Class Diagram	Described the UML diagrams and how they relate to use cases and quality requirements	Theo	Conrad
Current project	First draft of the current project version. With room to expand with future changes in deliverable	Alistair	Theo
Risk Assessment	Deleting two risks and replacing them with process risks. Then going through every risk and making it more explicit how we are mitigating the risk.	Alistair	Neela
Design Decisions	Justified design decisions based on quality requirements	Theo	Alistair
Verification	Acquired and found statistics to verify our quality requirements	Theo	Conrad
Risk Assessment	Rejigging the write up to better align with the new risks	Alistair	Neela
Key Drivers	Redo the KD table to fit the new SHs and QRs	Alistair	Conrad
Deployment diagram	Refined the deployment diagram and updated description	Neela	Alistair
GUI Diagrams	Reduced and condensed GUI diagrams section	Neela	Alistair
Key Drivers	Redoing the key drivers write up to fit the new table	Alistair	Neela
Project plan	Finalising project plan	Alistair	Theo
Current project	Writing about the unimplemented features as well as what went wrong in the development process.	Alistair	Conrad
Testing	Initial draft of the two testing sections	Alistair	Theo
DELIVERABLE 3			
Project Plan	Updating project plan for D3	Alistair	
Quality Requirements	Updated quality requirement table and refined descriptions	Neela	Theo
Quality Requirements	Updated write up about quality requirements	Neela	Theo
Testing	Acceptance testing write up	Alistair	Theo
Current Project	Updating entire C.P.V.	Alistair	
Testing	Added images of JaCoCo	Theo	Alistair

Testing	Rewrote unit testing section	Theo	Alistair
Product Overview	Re writing for consistency	Alistair	
Acceptance test	Refactoring acceptance test to fit new project state	Alistair	
K/D	Rewrote KD table	Conrad	Alistair
Current Product	Added information on features we added and considerations we made	Alistair	
Testing			
UML Diagrams	Rewrote and remade all diagrams and descriptions to accurately represent the altered structure of the program		

Business and System Context

Background

Canterbury is the fastest growing district in Aotearoa, with Christchurch city accounting for +52% of Canterbury's growth (Stats NZ, 2025). Additionally, the number of young people moving here is one of the highest in the country, with a 5.8% annual increase in 15–24-year-olds in 2024 (Burns, 2024). This growth has seen a large increase in restaurants, bars, and nightlife in the city, such as Riverside Market and The Terrace. A popular event, especially among young people, is to explore these places with a bar crawl (Fraser, 2025) .

A bar crawl, or pub crawl, is a journey to several different bars around a city, sometimes incorporating a theme, competition, or costumes. The bar crawl has likely existed for as long as pubs have, with the earliest organised pub crawl thought to be run by Oxford students in the 19th century (Pub Crawl Barcelona, 2026). Similar drinking events such as pub golf have been popularised recently by online influencers.

As well as being a fun way to explore a city, the pub crawl is much safer than other alternatives. House parties, with up to hundreds of young people, are notorious for becoming chaotic. Police are often called to out-of-control parties at student flats, such as in 2025 where six people were arrested at a house party in Riccarton (Smith, 2025). Similar incidents happen in other New Zealand cities, like the infamous incident in 2019 when a student was killed at a flat party in Dunedin (Sherwood & McNelly, 2019). Venues serving alcohol such as bars, restaurants, and nightclubs must have an on license which has many laws in place to prevent harm from consuming alcohol; minors and intoxicated persons

cannot be served, and food and transport options must be available (Ministry of Justice, 2026). These restrictions make drinking at bars a much safer option compared to visiting house parties.

In a large city with many bars, it can be difficult to create a route beforehand that is satisfactory; a pub crawl should be an appropriate distance, have well-spaced stops and be efficient, with minimal backtracking or looping. This can be solved with an application which automates the creation of a bar crawl route.

Product Overview

The product will be an application which helps design a bar crawl within Christchurch City. It simplifies the creation of a customised route between bars, reducing the barrier for users to choose this alternative as their alcohol related social event. This encourages people to explore the city by foot, increasing tourism and economic growth in the city. Furthermore, the app promotes safer drinking, as users will be by walking between places that must enforce responsible alcohol use, as opposed to uncontrolled house parties.

The user will be able to adjust various parameters to design the route; start bar, end bar, approximate distance, number of locations, bar themes (e.g. sports bars, nightclubs), specific bars they want to visit, and features they want in a bar (e.g. outdoor seating, pool table).

A route will be generated and displayed on a map. Stops along the way will be suggested, such as water fountains, public toilets, or fast-food places. These will not be used to determine the route but identified once created and shown to end user.

An account system will be implemented, allowing users to create, log into, and delete accounts. A signed in user is a member. Created routes can be saved to a member's account. Members can also publish routes to the app's 'Community Tab' where they can be viewed and loaded by all users. The Community Tab is a shared space where members publish their own routes for anyone to browse, load, and use. All users can view these published routes, and members can rate them to help others find the best pub crawl.

To ensure legal and responsible drinking, there will be a popup when first opening the app prompting the user to confirm they are over 18, the legal New Zealand drinking age. If they are not, then they cannot access the application. There will also be links to alcohol help services and safer drinking pages throughout.

Current Market

There are few applications capable of generating bar to bar routes and can be distinctly defined as either general mapping, organised pub-crawls, or exclusive pub-crawl generator. All 3 provide a route between bars, but all are catered to either a different selection of end users or neglect important quality requirements.

Competitors:

1. Map focused applications (Google Maps, Apple Maps, and Waze)

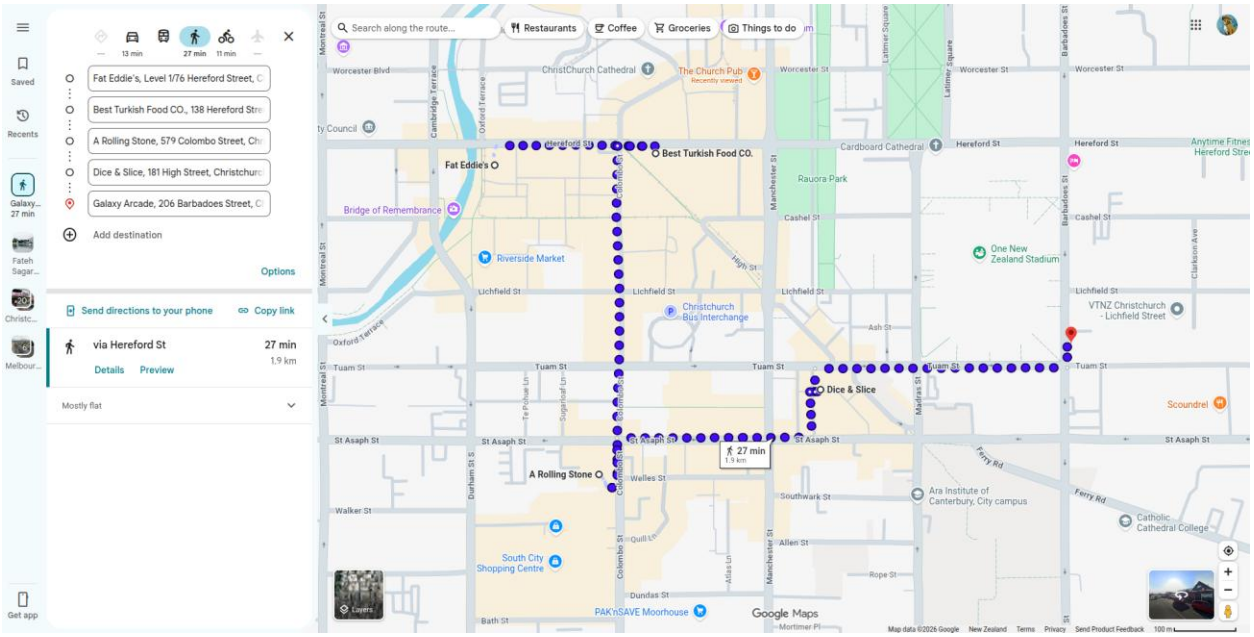


Figure 1: Screenshot of generated route from Google Maps

General navigation apps are the strongest in the current market due to a combination of ease of access and reliability. An example from a competitor, Google Maps, is Figure 1. Map based applications prioritise detailed location information and 2-point route navigation. This focus provides exceptional bar information and clear point to point navigation but requires manual input of bars and re-ordering for effective route. 'Route Beer' will not have access to extensive location information but will excel in bar route generation.

2. Specific Bar-Hopping Apps (The Golden Mile (The Golden Mile, 2026), (PubCrawl, 2026)⍰⍰, (YourPubCrawl, 2026)⍰⍰, and (Dallmann, 2026)This leaves gaps for a low-feature free alternative, specifically targeted towards residents in Christchurch. Built

specifically around the intention of a pub-crawl, these apps excel in easily finding a bar route that can be tracked, saved and shared. Other features like live-location and mobile accessibility allow for users to update their friends on location and time, while also providing additional mobile interactions such as “Pub Golf” and “Total drinks tracker”. These extensive features charge a premium and only gather data for densely populated cities such as Auckland or New York.

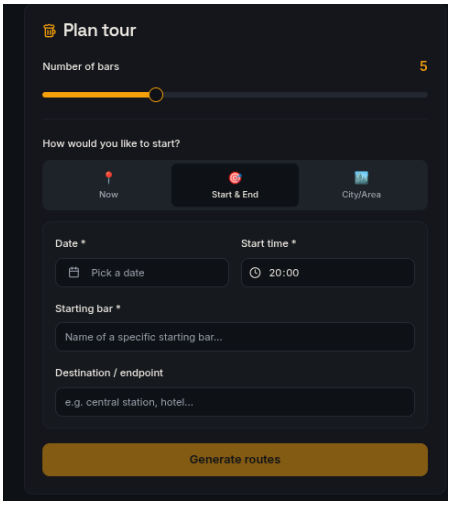


Figure 2: Snapshot from GoldenMile app

Feature 1: Input several parameters to generate a route. Source: https://goldenmile.bar/

Simple implementation of parametrised route generation but lacking proper support for Christchurch based areas. These features we found beneficial and decided to take inspiration when making our application

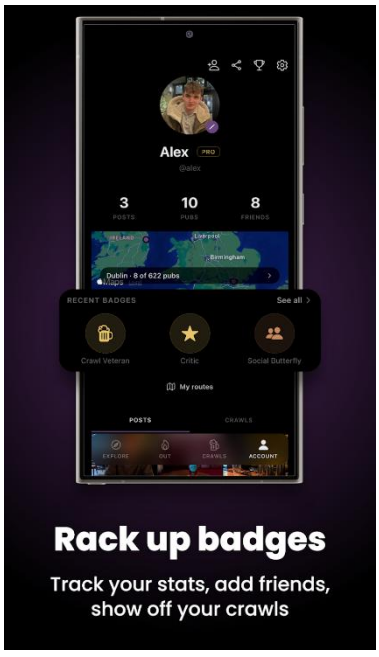


Figure 3: Snapshot from PubCrawl App

Feature 2: Social Interactions.

Add friends and communicated in-app. However, requires a subscription to use the service. A community aspect where users can publish, rate, and interact with other users’ routes would bring value to our application, but the reliance on a subscription-based service is not applicable to the application as it against the SENG202 project specification.

Feature 3: Community Organised Pub-crawls (Bar-Trek (Bar-Trek, 2024), Eventbrite (Eventbrite, 2026))

Community organised pub crawls are currently the weakest of our competitors. They offer excellent routes, detailed information and clear navigation, but at a steep cost of $25 to $50 per person (Eventbrite, 2026). This Community organised pub crawls are currently the weakest of our competitors. They offer excellent routes, detailed information and clear navigation, but at a steep cost of $25 to $50 per person (Eventbrite, 2026). This cost serves

as a major barrier for students, and lower income groups within the community, which our app will fill the gap with a free alternative.

Market Gap

While there are competitors which have aspects of bar-discovery or social outing, none combine the three main features to effectively generate bar-crawl routes in Christchurch. Easy route generation, detailed information of local bars, and fee free service. Every competitor has at least one of these features, but none of them have all of them together, and it is exactly this combination which sets our app apart

Easy and high-quality route generation is our key selling point. Rather than having user pick their own bars and manually select the ordering, like apps such as google maps, our app generates a complete walkable route through a few simple parameters. This turns what would be a lot of time planning the bar-crawl into a handful of click, removing the friction stopping people doing a bar crawl in the first place.

Detailed information about the local bars means our app is built on Christchurch specific knowledge and serves the city better than other apps such as YourCrawl. It provides you with a valid pub crawl but lacks any kind of additional features relevant to Christchurch. With our app, you can apply filters for certain bar themes or features you want, such as a pool table.

A no-cost implementation for the user is the final key feature which sets us apart. Where some apps such as PubCrawl require a premium subscription to generate a route with more than four locations, or any features, our app is free to use. This lowers the cost of choosing a bar crawl which makes it more accessible to tourists, students, and locals alike.

Together, these features form the foundations of the product through the development cycle and represent a market gap which no existing competitor fills.

System Context

There will be two main groups interacting with the system, as shown in

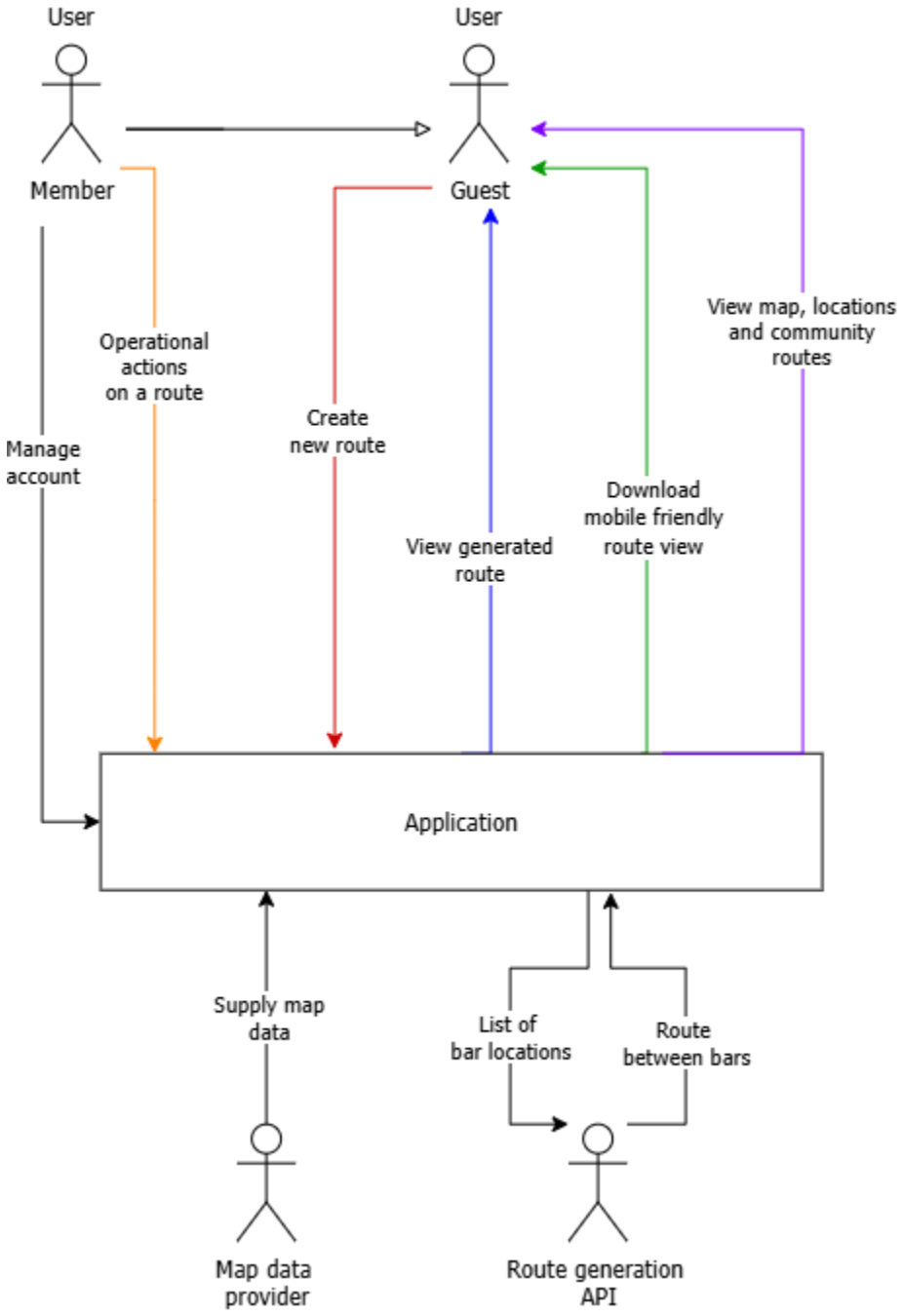


Figure 4: System Context diagram showing main system interactions

; users, which are split into guests and members, and external data providers, split into the map API and amenity API providers. Our users, upon opening the app, will have to verify that they are 18 or over. If they are not, the user will be informed that this application can only be

used by people over the age of 18, and the app will close. If the user is over 18, they will be prompted to either sign in, create an account, or continue as a guest. The application will then load the interactive Christchurch map interface, displaying bar locations as clickable map pins.

Both members and guests will have the option to create a route, where they will be required to select a start location. The user can adjust the number of bars and approximate distance from their default values of random, and any of the following parameters: end location, bar themes (e.g. sports bars, nightclubs), particular bars they want to visit, and toggleable bar features (pool tables, dart boards, outdoor areas).

The system will use these parameters to calculate an appropriate route fulfilling their specifications. Amenities near the route such as fast-food places, drinking fountains and public bathrooms will be shown on the map. The user will then be presented with the route details (walking time, walking distance, locations) and a visual representation of the route overtop the map interface. Both members and guests will have to option to export the route to an image file.

What distinguishes members and guests is persistence and participation. Where a guest's route can only be stored for the current session, members have the option to save routes to their account, which they can later load, leave ratings on these routes, and publish them to the community page. There will be a community page, where all users can view and load routes published by members. However, only members can upload routes and leave ratings on published routes.

External data providers will provide the system with necessary information, the map data provider giving the map data of Christchurch, and amenity APIs will provide location information for amenities such as drinking fountains and public bathrooms.

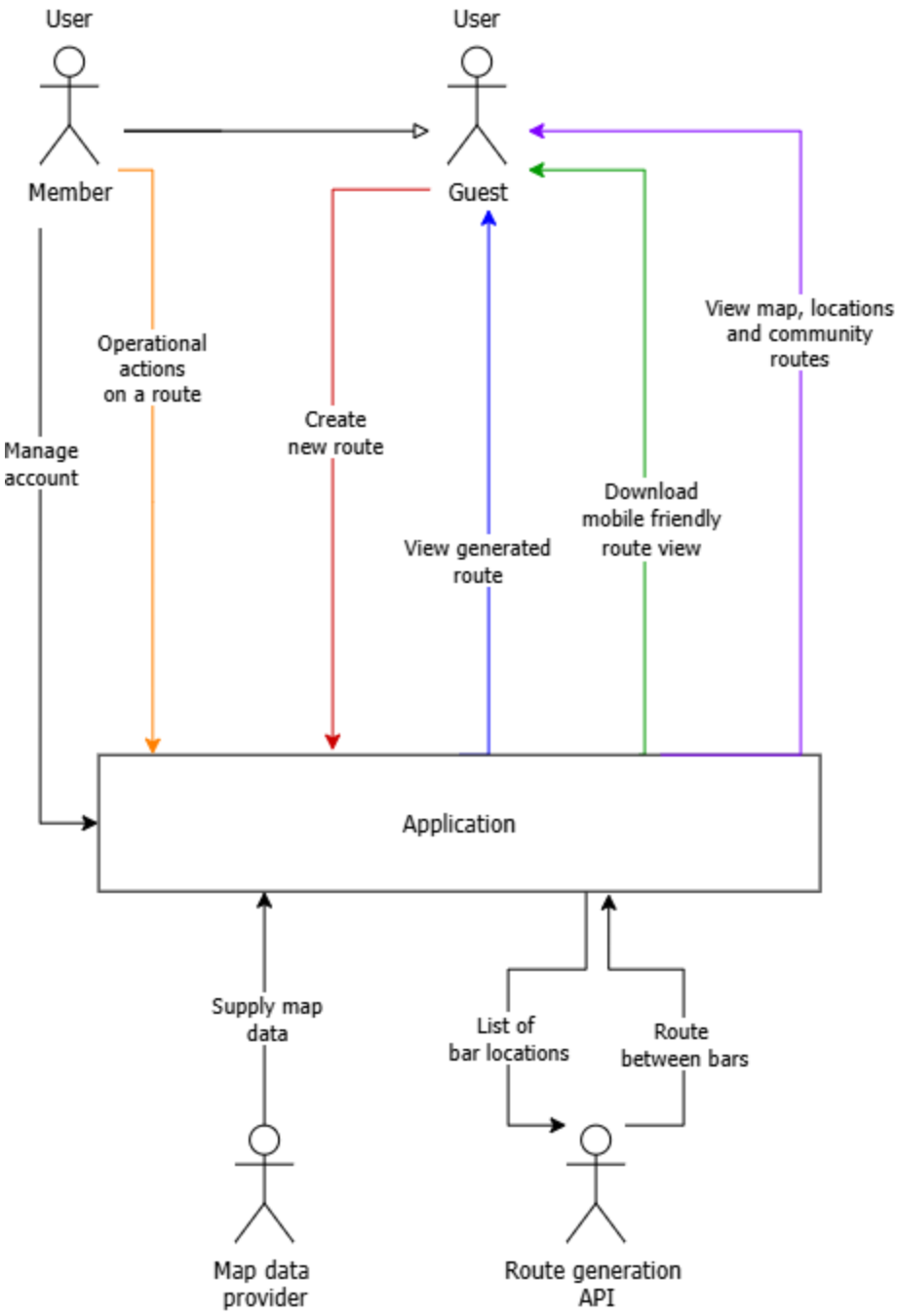


Figure 4: System Context diagram showing main system interactions

-

Stakeholders

Table 2 defines all stakeholders, their priority and relevant concerns to be considered in the designing of this application.

Table 2: Stakeholder analysis and priority ranking

ID	Name	Description	Priority	Concerns
SH_1	Non-Abstinent End-Users	Individuals over the age of 18 and participating in drinking related activities.	High	Want routes tailored to their specific input. They expect accurate and up to date venue information regarding opening times, location, and themes. They are looking for walkable routes.
SH_2	Developers	The student team designing, producing and maintain the product.	High	Clear requirements, technical feasibility, maintainable, and testable code. Aiming to create a high-quality product while meeting the SENG202 assessment criteria.
SH_3	Abstinent End-Users	Individuals over the age of 18 that do not participate in drinking related activities.	Med	When generating a route, their primary focus is on non-alcoholic related activities and beverages. The app provides them the ability to select activities to include in the route, such as pool, darts, or karaoke.
SH_4	Christchurch City Council	They are the local authority and the source of some of the open datasets the app relies on. Responsible for public spaces, licencing and city reputation.	Med	Their open datasets are used within license terms and not misrepresented. The app reflects the city responsibly and aligns with the council Policy, e.g. not having drinking in liquor ban areas.
SH_5	Pub owners / operators	Licensed venues that appear as candidates	Med	Being represented accurately (hours, location and features) is important for maintaining their brand's image. Publishing a route that includes their bar can increase foot traffic and spending.

SH_6	Alcohol regulatory and support boards	District licencing committee and organisations which aim to reduce alcohol related harm, e.g. Health NZ.	Med	Responsible promotion of alcohol, minimisation of harm. Not encouraging binge or high-risk drinking, and compliance with the Sale and Supply of Alcohol Act of 2012
SH_7	Ministry of Justice	The New Zealand Ministry of Justice is responsible for creating and enforcing laws and regulations at high levels.	Med	Compliance with laws and maintaining public safety. Enforces 18+ access and making sure the app does not present venues that would be law-breaking or disorderly.
SH_8	Third party API Providers	Map tile, routing service providers, plus any other relevant external services the app relies on.	Low	That the app implements their services within their terms and gives correct attribution. Needs are more contractual rather than about the purpose of the product.
SH_9	Mana Whenua	The local Māori iwi / hapū (Ngāi Tahu / Ngāi Tūāhuriri) who hold authority and take responsibility over the Christchurch area.	Low	Respectful representation of the whenua and wāhi tapu. Creating routes that do not go through significant sites or provide a user warning about the importance. They also care about the wider alcohol related harm in the community.
SH_10	Residents who reside on routes	People living on or near the streets that generated routes pass along	Low	Repeated routing down the same street concentrates noise, litter, and disorderly behaviour late at night. Wants routes distributed to avoid overloading particular streets.

Priority is scored on a scale of low, medium and high where it regards how important a certain group's needs are to be met by the project. There are three factors which influence the score for every entry; importance to the project's success, overall consequence of neglect, and likelihood that neglecting them could force the application down (functionally or reputationally). These factors were combined into priority groupings and can be investigated with a similar viewpoint.

Stakeholders with high priority have needs that are inseparable from the product's purpose. Failing to meet these stakeholder's needs will result in an unsuccessful product. Their needs shape how the app will look and function. Stakeholders with medium priority are important for legitimacy, safety or compliance. However, they may not directly determine whether people use the app. The app may still function without their needs met but will negatively impact the quality. Stakeholders with low importance still have some level of stake in the project but significantly less than other stakeholders. These stakeholders are largely contractual or externality based, where neglection causes minimal, recoverable impacts.

Non-abstinent end users (SH1) stand out as the clear most important stakeholders. This is because the app's entire premise depends on their needs and whether they will use it or not. It depends on their experience and whether they will return to use it again and recommend it to others. This holds for abstinent end users (SH_3) as they will use the app in a similar way, however along with making up a smaller portion of end users, the primary purpose of the application is less so aimed at them. There are multiple regulatory bodies which span alcohol consumption, data use and harm minimisation. They carry real, legal, and reputational implications but do not necessarily use the app. Developers (SH_2) also have a high priority because of the role they play in creating a high-quality application and making sure people use the app.

One thing to note with this ranking, is that low priority within our stakeholders does not necessarily mean low importance in general. Mana Whenua (SH_9) concerns in particular carry large ethical weight, higher than the score might suggest. It is just that in the context of our project, Mana Whenua do not have significant impact on the success or failure or our application. Alcohol-harm considerations also play a part in multiple stakeholders rather than just being confined to just one; the priority of them is essentially "spread" so is lower on each stakeholder. The scale is to consider trade-offs under constraint, not to disregard stakeholders who might hold a lower ranking than others.

Use Cases

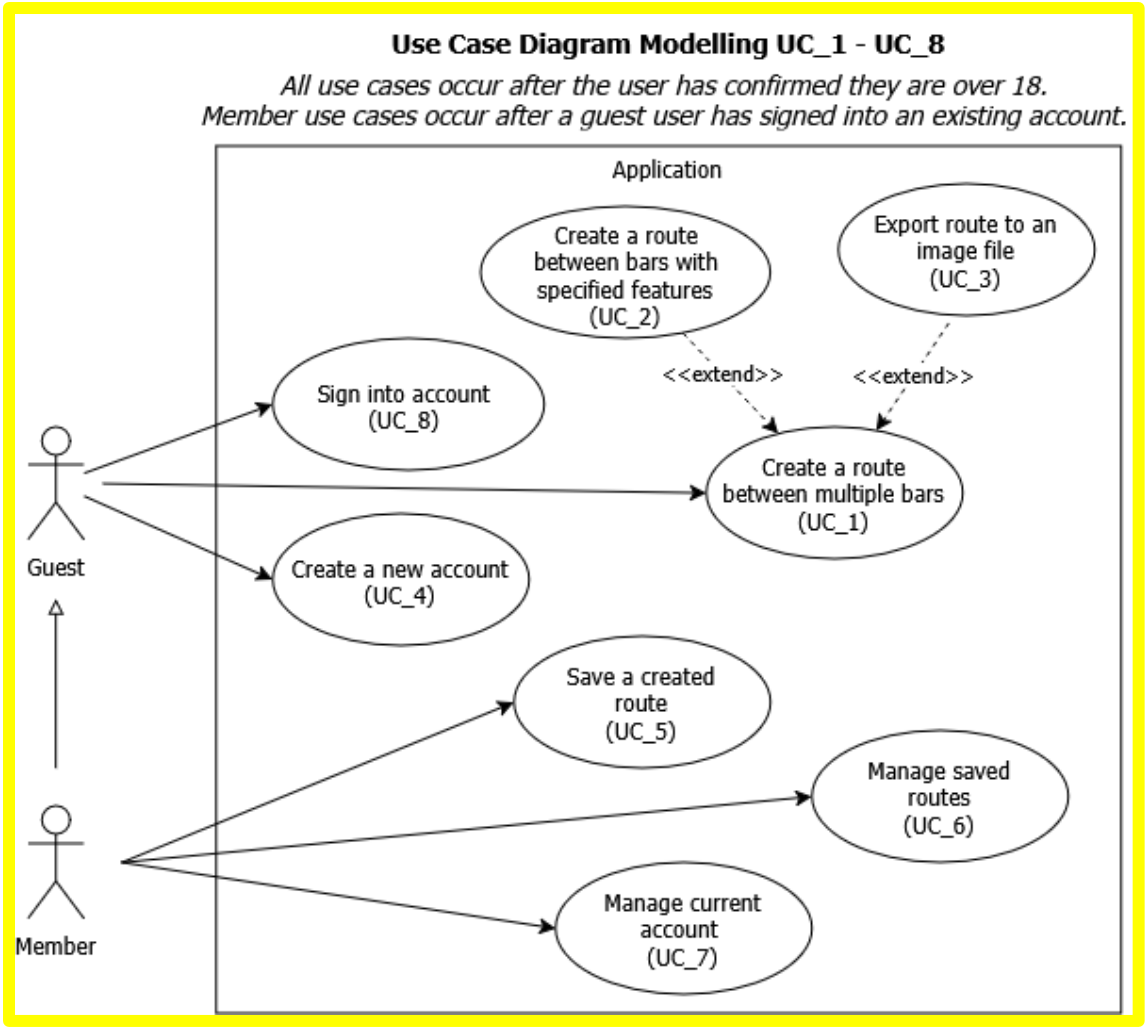


Figure 5: Use Case diagram showing use cases 1 through 8

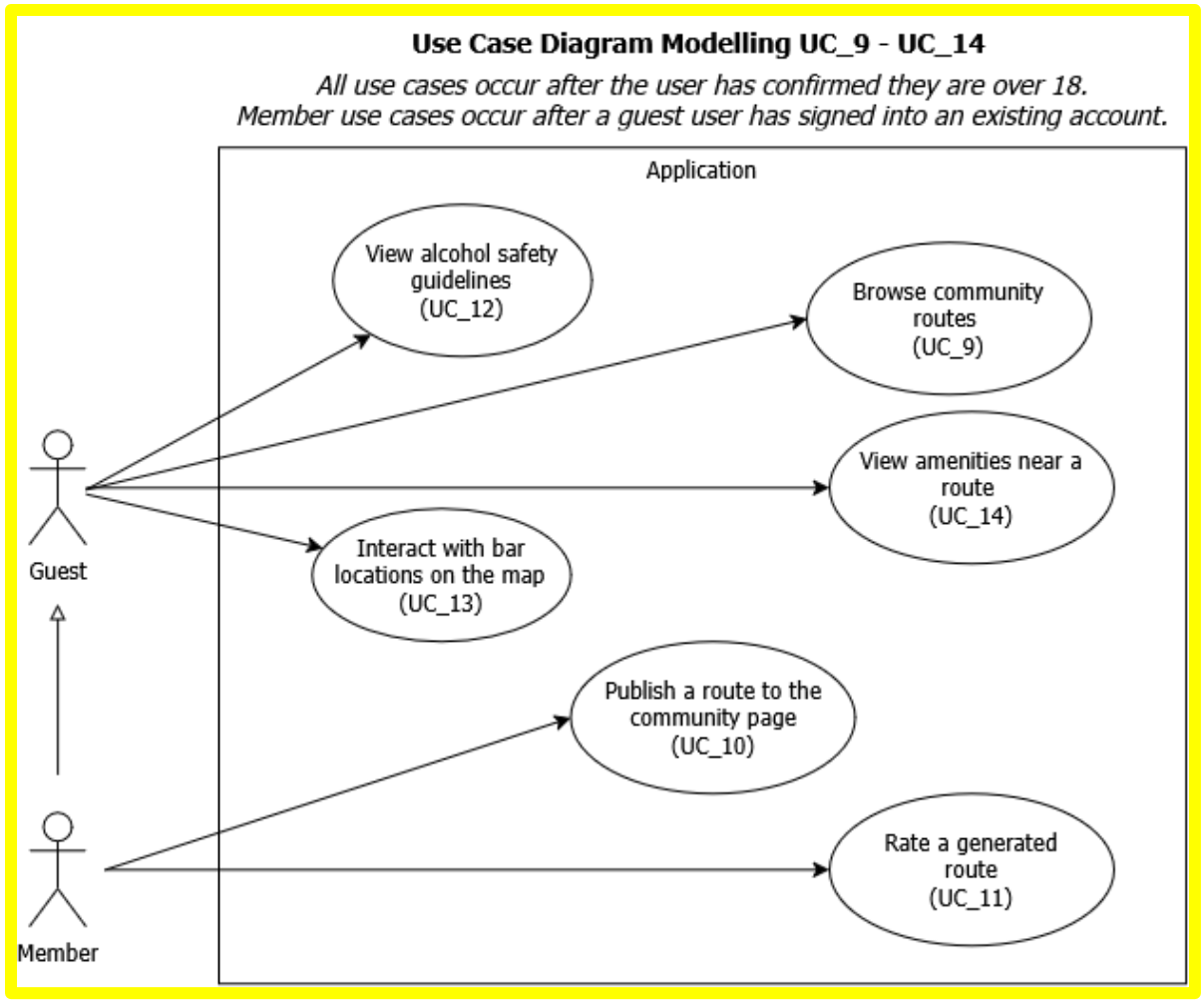


Figure 6: Use Case diagram showing use cases 9 through 14

Tables 3 to 11 demonstrate the 8 highest priority use cases, and their different flows.

Table 3: Use Case 1 analysis with blue sky, alternative and exceptional flows

UC_1		Generate a route between multiple bars with default parameters.
Priority		*High*
Actors		Non-Abstinent End Users (SH_1)
Scenario: Blue Sky	**Precondition**	*The app has launched and successfully loaded all location data and is displaying bars on the map. A tab with all parameter fields is displayed.*
	Flow	*The user fills in the 'Start Location' field with a valid input (a location from the bar Json) and clicks the generate button.*
	Postcondition	*A route is generated starting from the given start location which goes through 5 bars and displayed on the map. Fast-food places, drinking fountains and public toilets near the route are displayed on the map. Details of each bar in the route (including their features) and the total walking distance and time is displayed.*
Scenario: Exceptional	**Precondition**	*The app has launched and successfully loaded all location data and is displaying bars on the map. A tab with all parameter fields is displayed.*
	Flow	*The user enters an invalid start location (i.e. not a location in the bar Json), and clicks the generate button*
	Postcondition	*Error message appears, informing the user "Please pick a start location from the available bars."*

Table 4: Use Case 2 analysis with blue sky, alternative and exceptional flows

UC_2		Generate a route between multiple bars based on specifically defined parameters.
Priority		*High*
Actors		Non-Abstinent End Users (SH_1), Abstinent End Users (SH_3).
Scenario: Blue Sky	**Precondition**	*The app has launched and successfully loaded all location data and is displaying bars on the map. A tab with all parameter fields is displayed.*

	Flow	*The user fills in the 'Start Location' with a valid input. Then select the route to have exactly 5 bars. They select the checkboxes to include bars with a pool table and an outdoor area. They additionally select the bar theme 'Irish' and click the generate button.*
	Postcondition	*A route is generated between 5 bars, where there is at least one; bar classified as 'Irish', bar with a pool table, and a bar with an outdoor area. Fast-food places, drinking fountains and public toilets near the route are displayed on the map. Details of each bar in the route (name, address, bar order number) and the total walking distance and time is displayed.*
Scenario: Exceptional	**Precondition**	*The app has launched and successfully loaded all location data and is displaying bars on the map. A tab with all parameter fields is displayed.*
	Flow	*The user fills in the 'Start Location' field with a valid input. They then set the numbers of bars to 3, and select the themes; "Rooftop Bar", "Cocktail Bar", "Sports Bar", and "Nightclub". Then they click the generate button.*
	Postcondition	*The user is notified of an invalid input and receives the error message: "Couldn't fit your selected theme(s) into a route. Try adjusting the filters."*

Table 5: Use Case 3 analysis with blue sky, alternative and exceptional flows

UC_3		*Export route to an image file*
Priority		*High*
Actors		Non-Abstinent End Users (SH_1), Abstinent End Users (SH_5).
Scenario: Blue Sky	**Precondition**	*A user has generated a valid route. An 'Export Route' button is made available.*
	Flow	*The user presses the export button. The export window launches, and a preview of the route as an image is shown. The user clicks the 'Export to PNG' button.*
	Postcondition	*The route can be downloaded to a file of the user's choosing, along with customizing the file name*
Scenario: Alternate(s)	**Precondition**	*A user has loaded a route from the 'Community' tab. An 'Export' button is made available.*

	Flow	*The user presses the export button. The export window launches, and a preview of the route as an image is shown. The user clicks the 'Export to PNG' button.*
	Postcondition	*The route is successfully exported as an image file and saved to a file of the user's choosing and the user can customize the name of the downloaded file.*
Scenario: Exceptional	**Precondition**	*A user has successfully generated a new route. An 'Export to PNG' button is made available.*
	Flow	*The user presses the export button. They then name the route which has the same name as an image in the user's download folder.*
	Postcondition	*The user is notified that this file already exists and is asked to rename the route.*

Table 6 - Use Case 4 analysis with blue sky, alternative and exceptional flows

UC_4		*Create a new account*
Priority		*Med*
Actors		*Non-Abstinent End Users (SH_1), Abstinent End Users (SH_3).*
Scenario: Blue Sky	**Precondition**	*The app has successfully launched, the user has selected that they are over 18 years old, and the login window is open.*
	Flow	*The user selects the 'Create Account' button. Then the user puts in a valid username (between 5-15 characters, no whitespace, '.', or '_'), a valid password (between 8-16 characters, letters, numbers, or any of '!@#$%^&*'), then confirms the password. The user then selects the "Create New Account!" button.*
	Postcondition	*The account creation is accepted, the password is encrypted using a jBCrypt library* (MindRot, 2026) *and stored alongside the username in the remote database. The user is then presented with the main window and can access all the features that the app holds.*
Scenario: Exceptional	**Precondition**	*The app has successfully launched, the user has selected that they are over 18-year-old, and the login window is open*
	Flow	*The user puts in a username which is already registered in the application system and selects the "Create Account" button.*

	Postcondition	*The user is denied an account creation and is prompted with a statement saying an account is already registered with that username. They remain on the login window and can retry creating an account but with a different username.*

Table 7 - Use Case 5 analysis with blue sky, alternative and exceptional flows

UC_5		*Save a created route*
Priority		*Med*
Actors		Non-Abstinent End Users (SH_1), Abstinent End Users (SH_3).
Scenario: Blue Sky	**Precondition**	*A member has signed into an existing account. A valid route has been generated and is displayed on the map.*
	Flow	*The member presses the 'Save Route' button, with a name eg 'Bar Route 1' and a description eg 'Cool route'.,*
	Postcondition	*The route is saved to the member's account and a notification states 'Route successfully saved'. This route can now be viewed from the 'Saved Routes' tab.*
Scenario: Alternate(s)	**Precondition**	*A member has signed into an existing account. They have successfully loaded a route from the community page.*
	Flow	*The member presses the 'Save Route' button, with no name or description field filled*
	Postcondition	*The route is saved to the member's account and a notification states 'Route successfully saved'. This route can now be viewed from the 'Saved Routes' tab. It has the name 'Generated Route' and no description.*
Scenario: Exceptional	**Precondition**	*A user is not signed in. Required route fields have been filled out and a valid route has been generated and displayed on the map.*
	Flow	*The user presses the 'save route' button*
	Postcondition	*The user is notified that they cannot save routes when not signed in. The user is then prompted to either sign in or create an account, and can press a button to redirect them to the login page.*

Table 8 - Use Case 6 analysis with blue sky, alternative and exceptional flows

UC_6		*Manage saved routes*
Priority		*Med*
Actors		Non-Abstinent End Users (SH_1), Abstinent End Users (SH_3).
Scenario: Blue Sky	**Precondition**	*A member has signed into an existing account with saved routes, and navigated to the 'Saved Routes' tab.*
	Flow	*The member selects a route and presses the 'Load Route' button*
	Postcondition	*The route is loaded to the map, and all relevant details are displayed*
Scenario: Alternate(s)	**Precondition**	*A member has signed into an existing account with saved routes, and navigated to the 'Saved Routes' tab.*
	Flow	*The member selects a saved route and clicks the 'Delete' button*
	Postcondition	*The route is removed from the member's account and no longer displayed on the 'Saved Routes' tab*
Scenario: Exceptional	**Precondition**	*A user is not signed in.*
	Flow	*The user presses the 'Saved Routes' tab.*
	Postcondition	*The user is notified that they must be logged in to view saved routes. The user is then prompted to either sign in or create an account, and can press a button to redirect them to the login page.*

Table 9 - Use Case 7 analysis with blue sky, alternative and exceptional flows

UC_7		*Manage current account*
Priority		*Med*
Actors		Non-Abstinent End Users (SH_1), Abstinent End Users (SH_3)
Scenario: Blue Sky	**Precondition**	*A member has signed in and navigated to the 'Account Information' tab.*
	Flow	*The member clicks the 'Sign out' button.*
	Postcondition	*The member is logged out. The user is then taken back to the login screen and is prompted to sign in or continue as guest*
Scenario:	**Precondition**	*A member has signed in and navigated to the 'Account Information' tab.*

Alternate(s)	**Flow**	*The member clicks the 'Delete account' button. A popup appears with a yes and no button, asking if the member is sure they want to delete their account. The member clicks yes.*
	Postcondition	*The member's username and password are deleted from the database, as well as all routes associated with their account, including published routes.*
Scenario: Exceptional	**Precondition**	*A user has launched the app and continued as a guest.*
	Flow	*The user has navigated to the account details page.*
	Postcondition	*The user is notified that they must be signed in to see account details.*

Table 10 - Use Case 8 analysis with blue sky, alternative and exceptional flows

UC_8		*Sign into account*
Priority		*Med*
Actors		*Non-abstinent end users (SH1), Abstinent end users (SH_2)*
Scenario: Blue Sky	**Precondition**	*The app has successfully launched, the user has selected that they are over 18 years old, and the login window is open.*
	Flow	*The user selects 'Sign In' with the username field filled with a registered username, and the password field filled with the matching password*
	Postcondition	*The user is transported into the main window, and their saved routes are loaded into the 'Saved Routes' tan*
Scenario: Exceptional	**Precondition**	*The app has successfully launched, the user has selected that they are over 18 years old, and the login window is open.*
	Flow	*The user selects 'Sign In' with the username field filled with a registered username, and the password field filled with a non-matching password*
	Postcondition	*The user is informed that the password is incorrect and they remain in the login window*

Table 11 - Use case 9-15 priority ranking and stakeholder association

Use Case #		Priority	Actors
9	Browse community routes	Low	Non-Abstinent End-Users (SH_1), Abstinent End-Users (SH_3), Pub owners / operators (SH_5)
10	Publish a route to the community page	Low	Non-Abstinent End-Users (SH_1), Abstinent End-Users (SH_3), Pub owners / operators (SH_4).
11	Rate a generated route	Low	Non-Abstinent End-Users (SH_1), Abstinent End-Users (SH_3), Pub owners / operators (SH_4).
12	View alcohol safety guidelines	Low	Christchurch City Council (SH_4), Alcohol regulatory and support boards (SH_6), Ministry of Justice (SH_7)
13	Interact with bar locations on the map	Med	Non-Abstinent End-Users (SH_1), Abstinent End-Users (SH_3), Pub owners / operators (SH_5).
14	View amenities near a route	Low	Non-Abstinent End-Users (SH_1), Abstinent End-Users (SH_3)

App Entry

When launched, the application will display a page where the user must confirm that they are over the age of 18. The “Sign into an account” button will let the user continue to the app as a member (UC_8), granting them extra abilities such as saving and rating routes. All use cases must take place after the age check has been completed, and after the user has either signed in or continued as a guest. Figure 5 models the base app functionality, how users will manage accounts, create routes and interact with the map. Figure 6 models the extended functionality involving accessing and publishing to the community page alongside rating routes and other use cases

Quality Requirements

Table 12 is an evaluation of all quality requirements the application should fulfil to satisfy the project and stakeholder requirements.

Table 12: Quality requirement introduction, explanation and stakeholder relation

ID	Title	Description	Steps to Achieve	Stakeholders
QR_1	Performance	Routes should generate and load quickly, map loading and app navigation should be fast.	New route generation should have a user perceived response time (UPRT) under two seconds, or under four seconds with loading animation. Saved routes should have a UPRT under two seconds, industry standard. Switching between sign in/create account window and main window should have a URPT under two seconds(SEO Villas Private Limited, 2024)	Non-Abstinent End Users (SH_1), Abstinent End Users (SH_3).
QR_2	Robustness	The application should behave correctly and predictably, including when it receives invalid input or something goes wrong. Methods should validate their arguments and handle invalid ones in a defined way rather than crashing or returning incorrect output. The application should also recover gracefully when a route cannot be generated. Because of the large number of parameters available for route customisation, it should be clear to the user exactly what parameters are causing issues if a route cannot be generated.	Automated testing should have 85% or greater instruction and branch coverage for non-GUI classes (Zyaparova, 2025). 80% instruction and branch coverage for all classes.100% of automated tests pass before code is merged into the main branch. Zero uncaught exceptions or crashes when invalid route parameters are entered during testing. When no valid route can be produced (e.g. constraints are impossible), the user receives a clear message rather than an empty or incorrect result.	Non-Abstinent End Users (SH1), Developers (SH_2), Abstinent End Users (SH_3).

QR_3	Usability	The application should be easy to read, understand and navigate. Text, labels (e.g. on route parameters) and error messages should be simple to comprehend, navigation should be kept simple, and errors should be detected, handled consistently and presented to the user in a friendly way.	100% of text should be minimum 16px in size, larger for titles, font must be dyslexic friendly (Ako Aotearoa, 2023), 85% of people should be able to read and comprehend content of application (Australian Government, 2021), Colours and contrasts should fit WCAG guidelines: 4.5:1 contrast minimum for all text, 3:1 contrast for all else. (W3C, 2025). Should pass colourblind simulations for protanopia, deuteranopia and tritanopia. Maximum three clicks to navigate to any page on the home window of the application, Average Task Completion Rate 90%, Average System Usability Score (SUS) 80 (Sauro, 2012). 100% (Tsipenyuk, Chess, & McGraw, 2026).	Non-Abstinent End Users (SH_1), Abstinent End Users (SH_3)
QR_4	Maintainability	Planning for future expansion by keeping code extendable and minimally coupled, ensuring code is tidy and readable. Enforce separation of concerns, making each class have a single purpose.	The codebase contains no cyclic dependencies. It passes our style checker with zero violations, and all methods except getters, setters, and tests have JavaDoc comments. Every change is peer reviewed before merging. Follow the Model View Controller (MVC) structure to reduce issues in future refactoring and development.	Non-Abstinent End Users (SH_1), Developers (SH_2), Abstinent End Users (SH_3).
QR_5	Data accuracy	Route and venue location data shown to users should be correct, current and consistent with its source. Map should be correct and route paths, distances and estimated time should be accurate.	Use trustworthy APIs to source location data, data is checked and validated to ensure false locations are removed, ensure distance and estimated time for generated routes is accurate. Use an up-to-date map API and accurate route generation API.	Non-Abstinent End Users (SH_1), Abstinent End Users (SH_3), Christchurch City Council (SH_4), Pub owners/operators (SH_5).

QR_6	Aesthetics	Application should look visually pleasing and consistent.	Visual design should have 80% customer satisfaction score (CSAT) from independent reviews from stakeholders (Perspective AI, 2026). Elements should use shared styles from a central style sheet to keep colours, fonts and other GUI elements consistent.	Non-Abstinent End Users (SH_1), Abstinent End Users (SH_3).
QR_7	Account Security	Accounts should be securely stored within our database	100% of user passwords are hashed with a salt using a secure algorithm, and no passwords appear in plaintext in the database, logs or source code. API keys and database credentials are held outside the repository, in environment variables.	Non-Abstinent End Users (SH_1), Abstinent End Users (SH_3), Ministry of Justice (SH_7).
QR_8	Legality	Data should be used according to API terms and conditions, follow similar guidelines to licensed premises regarding safe alcohol consumption.	100% of API data usage should comply with provider terms and conditions, application should promote to user food and water, and driving alternatives. Users should be informed of the dangers surrounding alcohol and provided quick access to support services.	Developers (SH_2), Christchurch City Council (SH_4), Alcohol regulatory and support boards (SH_6), Ministry of Justice (SH_7), Third party API Providers (SH_8).

Performance QR_1 is our highest priority because a user will lose interest and leave the app if they are having to wait a significant amount of time for every route to generate. Our primary goal is to simplify generation and customisation of routes, allowing many specific parameters to be customised and encouraging reuse of the route generation feature. Mild delay would be acceptable if the user only had to experience it once, but this delay would become more consequential if present every time a user regenerates a route.

Robustness comes second because route generation has many customisable parameters, so invalid input and impossible constraints are likely. The application must handle these without crashing and tell the user exactly which parameters caused the problem. Otherwise, the user could be confused, not know how to fix their route, or may think there are issues with the app.

Usability is third, as it determines whether people can use the application at all. An estimated 10% of New Zealanders are dyslexic (Cole, 2021), and approximately 4.5% of people have a colour vision deficiency (Greenlane Penrose Optometrists, 2019), showing the importance of catering towards this demographic of our end users. Readable text sizes and fonts, strong contrast and colour-blind friendly design will allow more people to use our app and enjoy a bar crawl. Users are also more likely to continue using an application that is easy to understand and navigate (UIUX Design School, 2025). Additionally, there is a likelihood for users to be intoxicated when using the app, further increasing the importance of making it easy to use

Maintainability follows, keeping the code loosely coupled and following the Model View Controller structure so the application can be extended and developed with ease. Data accuracy comes next, ensuring that venues, maps, distances and estimated times come from trustworthy APIs and are validated before being shown to the user. Aesthetics ranks lower because it matters less than the functionality of our application, however a consistent and visually pleasing consistent design will increase the satisfaction of users and encourage them to return.

Account security requires all passwords to be hashed with a salt, and API keys and database credentials to be kept outside the repository, so user data cannot be recovered if the database is compromised. Account security is low on our application's quality requirements, as we will specify to users to not use any real credentials, and the only data being stored about users is their credentials, saved routes and route ratings, none of which pose a real security threat or safety risk if compromised.

Legality requires data to be used within API providers' terms and conditions, and breaching these requirements would likely involve loss of resource access, directly affecting our app. Considering alcohol being the focus of our application, it is essential to follow national requirements surrounding promotion of safe alcohol use and harm mitigation. However, the likelihood of our application having legal altercations is unlikely; much of the legal responsibility will fall to end users and bar operators, which is why it is low priority in our quality requirements.

Design decisions

Route generation is the primary feature that impacts our application's performance (QR_1), and considering that performance is our highest quality requirement, optimisation of this algorithm is a high priority. A brute force algorithm approach to generate routes between bars would result in a time complexity of approximately $O(R \cdot n^2 \cdot m^2)$, where R is number of candidate routes per call, n is number of locations to visit, and m is number of bars in Christchurch; extremely slow and inefficient. Our chosen approach for the bar choosing algorithm involves use of a pre-calculated distance matrix and binary recursion with semi-greedy searching. Storing the distance from each bar to every other bar in a matrix makes it incredibly efficient to access and compare distances for any combination of locations. This optimisation results in an asymptotic complexity of approximately $O(R \cdot n^2 \cdot m)$, and the brings the URPT down from tens of seconds to under one second when successfully generating route. We also use a modularity pattern for loading new FXML files; it is more efficient and tidier to inject smaller FXML systems into the active window, therefore we did this to decrease waiting time between swapping windows or opening new sections.

We used JUnit tests to ensure QR_2 robustness because of the coverage testing with JaCoCo and clear test separation. This is expected to reduce overall time spent testing by approximately 30% (Amaros, 2025). We used standard JUnit tests for internal functionality with models, services, data, and exceptions. For user-experience testing, we used TestFX, which simulates user input of the application, because it allows us to test that the user is receiving feedback to incorrect inputs or errors, particularly in route generation.

Our usability quality requirement (QR_3) is influencing multiple design decisions, such as having a base font size of 16px, and ensuring that colour interactions (such as coloured text on a coloured background) have high contrast and visibility. To maximise readability and comprehensibility, we are using a sans-serif font (Kanit) for regular text and are keeping labels and text simple. This should "definitely make it easier to read" (Blake Latimer, personal conversation September 16, 2026). The use of a split window design for our main screen, consisting of a tab pane holding main functionality and the map to the right, further improves our application's usability. This allows the user to always have access to the map and navigate to any other page in a single click. The use of listeners allows for instantaneous and responsive error communication to the user, and other errors are identified and reported to the user as soon as they occur. All error reports include specific information on their cause, accompanied with advice for resolution.

We used class extensions throughout our code such as Bar, FastFood, PublicToilet, and DrinkingFountain all being descendants of the Location class. This allowed us to reduce

duplicate code and write methods which can be shared across different classes by using the parent type. Multiple elements in the GUI code as well share common attributes, the route and bar cards have base FXML files which get adjusted at runtime to fit any varying requirements, such as cards for saved routes not showing usernames and ratings, and community route cards discarding unneeded buttons. Shared CSS files with CSS classes also allow for easily adjustable and consistent colours and designs. This means any design or functionality changes are immediately reflected across all instances, and switching cards between forms is seamless.

Verification

Route generation time for QR_1, performance, was verified by measuring time for “No Route Generated” error popup to appear, averaging 3.7s URPT, and successful route generation, averaging 0.6s. Route loading time is effectively instantaneous, <0.2s URPT. Switching from sign-in to map window and map window to sign-in averaged 0.9s and <0.2s URPT respectively. Robustness, QR_2, we verified with a JaCoCo generated coverage report which was at approximately 85% non-GUI related coverage. For QR_3, usability, the application had an average SUS score of 80.4 (Table 155), and Average Task Completion Rate of 87% (Table 16). Readability was verified by asserting all fonts were 16px or bigger and sans-serif, which is considered dyslexia-friendly. Titles were observed to be larger, and all subheadings and general text was appropriately sized. 92% of people found text in our application easy to read and comprehend as seen in Table 17.

Key Driver Analysis

To decide the importance of each quality requirement (QR), each stakeholder (SH) allocated a percentage of their concerns across each QR. Each SH has 100 points distributed across all the QRs. Only a select number of stakeholders were used (SH_1– 4, SH_6, SH_7) as they either directly use the application or have the potential to terminate the app. A weighting of 0-3 were given to each stakeholder based on their priority in Stakeholder table. As Christchurch City Council (SH_4), Alcohol regulatory and support boards (SH_6) and Ministry of Justice (SH_7), are not end-users, their weighting is scaled down to 0.33 as their impact on the development of the app is indirectly proportional to Legality (QR_8).

Name of Stakeholders	**Weighting**	**Performance (QR_1)**	**Robustness (QR_2)**	**Usability (QR_3)**	**Maintainability (QR_4)**	**Data accuracy (QR_5)**	**Aesthetics (QR_6)**	**Account Security (QR_7)**	**Legality (QR_8)**
Non-Abstinent End-Users	**3**	**25**	**25**	**25**		**15**	**10**		
Developers	**2**	**5**	**10**		**60**	**15**			**10**
Abstinent End-Users	**2**	**25**	**20**	**25**		**15**	**15**		
Goverment Bodies (SH_4, SH_6, SH_7)	**0.33**								**100**
Total Sum of Points Given:		**135**	**135**	**125**	**120**	**105**	**60**	**0**	**53**

Figure 5 - Filled Key Driver Table

The most important key drivers identified were Performance (QR_1), Robustness (QR_2) and Usability (QR_3) with 135, 135 and 125 points respectively. Each scored highly for end users, as they all serve to improve on the user's interactive experience. Maintainability (QR_4) with 120 points being solely from Developers (SH_2), skew the table as end users are not directly impacted by the maintenance evolution of the product. Both Data Accuracy (QR_6) and Aesthetics (QR_7) were weighted on the lower half as although end users and developers are both impacted, they have less significance than other foundational requirements.

Classification: In-Confidence

Acceptance Tests

Table 13 details the acceptance tests necessary to implement the defined use cases.

Table 13 - Acceptance Tests of various use case flows, in Given-When-Then format

ID	Use Case	Scenario	Steps	Acceptance Criteria	Weight
AT_1	Generate a route between multiple bars with default parameters (UC_1)	Generate a route using minimal input values.	Given: The user is on the main screen When: The user selects generate route with a valid starting location Then: A random number of bars between 3-9 are shown on the map interconnected with total walking distance and total walking time.	Route contains exactly between 3-9 distinct venues. Visual route is rendered between each point. Total walking time and distance are calculated and displayed.	High
AT_2	Generate a route between multiple bars based on specifically defined parameters (UC_2)	Create a route while having filters applied.	Given: The user is on the main screen When: the user selects 'Create Route' with all the available fields filled out (start, end, distance, number of bars, themes, and features), Then: a route is generated meeting the criteria, and the route is displayed on the map.	A route is displayed showing a path between the selected locations with the filters selected all applied. The route returned is within 20% of the approximate distance set by the user.	High
AT_3	Save a created route (UC_5)	Save route feature	Given: the user is logged in, and a route has been created on the home screen, When: the user selects "Save route", Then: the route is added to their saved routes	The route is saved to their account. The saved route will appear and can be viewed in the 'Saved Routes' tab.	High

AT_4	Export route to an image file (UC_3)	Exporting a route	Given: the user has a route displayed on screen, When: the user selects "Export to PNG", Then: the user is the presented a preview to their route as a PNG file. They can then select where it will be downloaded to and what the file will be called	The route is locally exported as a PNG file in the user's selected directory.	High
AT_5	Manage saved routes (UC_6)	View saved routes feature.	Given: a user is signed in with one or more saved routes, When: the user selects the 'Saved Routes', Then: every route associated with the account is listed, each showing its key filters.	All routes associated with this account appear in the list with their key details	High
AT_6	Manage saved routes (UC_6)	Loading routes from a user's saved routes	Given: a user has signed into an existing account and is in the 'Saved Routes' tab When: the user selects the "Load" button on one of their routes, Then: the user can see their saved route shown the map with the bar fields on the side of the screen.	The details from this route are loaded into the corresponding fields on the map and the route is displayed on the map	High
AT_7	Create a new account (UC_4)	New user creating an account	Given: the user is on the login window and has selected create new account, When: the user selects "Create Account" with a unique username and valid password Then: the account becomes registered in the app database, and the user is directed to the home page.	A new account is created under the given details, and the user is redirected to the main page signed into this account	High
AT_8	Sign into an account (UC_8)	User logging into a previously created account.	Given: a user is on the login window When: the user selects the "Sign In" with a registered username and password, Then: the user will load into the main window of the application.	The saved routes associated with this account are loaded into the 'Saved Routes' tab, and the main window is launched.	High

AT_9	Interact with bar locations on the map (UC_13)	Browsing the map.	Given: the user is on the map screen, When: the user pans and zooms the map across the Christchurch CBD, Then: the user is presented with venue pins overlaid on a consistently rendered map.	The map and pins render at various zoom levels. There are no missing tiles, clicking on a location's pin displays that location's name, address and opening hours.	High
AT_10	View alcohol safety guidelines (UC_12)	Safety popup on application start	Given: The user has launched the app When: The user clicks (I am 18+) Then: Transition to login page.	The user is then presented with the login window	High
AT_11	View alcohol safety guidelines (UC_12)	Safety popup on application starts where user is under 18 years old	Given: The user has launched the app When: The user clicks (I am not 18+) Then: A message appears telling the user they must be 18+ years of age to use the app	App terminates after a message telling the user they cannot use the app under 18 years of age	High
AT_12	Save a created route (UC_5)	User is not logged in	Given: the user is browsing as a guest, and a route has been generated, When: the user selects "Save Route", Then: Nothing will happen and the user will be able to see that they cannot save a route while not logged in	The user is notified that they cannot save routes when not signed in, the save button is disabled	Med
AT_13	Manage saved routes (UC_6)	View saved routes as guest	Given: the user is browsing as a guest When: the user selects the 'Saved Routes' tab Then: the user is shown a button to sign in, and no route list is shown	A popup prompts the user to sign in and no routes are displayed	Med

AT_14	Manage saved routes (UC_6)	User having no saved routes.	Given: a user is signed in with no saved routes, When: the user selects the 'Saved Routes', Then: the message “No saved routes” is displayed, along with a button linking to the create route tab	The “No saved routes” message shows, with a button linking to the “Create route” tab	Med
AT_15	Sign into an account (UC_8)	User enters unregistered username	Given: a user is on the login window When: the user selects the “Sign In” button, with an unregistered username, Then: the user is notified invalid username or password	The user is notified that this account does not exist. They remain on the login page	Med
AT_16	Sign into an account (UC_8)	User puts incorrect password	Given: a user is on the login window When: the user selects the “Sign In” button, with a valid username but incorrect password, Then: the user is notified invalid username or password.	The user is notified that the password is incorrect. They remain on the login page	Med
AT_17	Generate a route between multiple bars based on specifically defined parameters (UC_2)	Filter combination matches no possible routes	Given: the user is on the main screen When: the user selects ‘Generate Route’ with too specific parameters (eg. Three bars, pool table, karaoke, live music, gaming machines, cocktail theme) Then: no route will generate and the user with receive a message telling them what went wrong with the parameters	The user with receive the message a message about what about their route failed. No route is generated	Med
AT_18	Interact with bar locations on the map (UC_13)	Toggle amenities on map	Given: a user is on the map screen, When: the user toggles amenities to be hidden, Then: the food, toilet, and drinking fountain pins are all hidden on the map	Toilet, food, and drinking fountain icons disappear from the map.	Med

AT_19	Manage current account (UC_7)	Log out of account.	Given: The user is logged in and on ‘Manage Account Tab’ When: "Log out" is clicked, then confirmed Then: Then the user is redirected to the home page.	User is in logged out state and returned to entry.	Low
AT_20	Create a new account (UC_4)	User attempts to create an account with a previously registered username	Given: a user is on the login window and has selected create new account When: the user selects “Create Account” with a username which is already registered, Then: the user is notified that the username is already registered and prompts them to log in or use a different username.	An error message pops up notifying the user that this username is taken. Remain on the login window.	Low
AT_21	Manage current account (UC_7)	Deleting an existing account	Given: the user is logged in and navigates to account settings, When: the user taps "Delete Account" and confirms via the confirmation prompt, Then: the account is no longer registered with the application and all routes associated with the account are lost, then the user is taken back to the login screen.	The account no longer exists. User is taken back to login page. Login with the deleted credentials fails, and all saved routes tied to that account are no longer retrievable.	Low
AT_22	Manage saved routes (UC_6)	Deleting a route	Given: the user has a saved route in 'Saved Routes', When: the user selects the route and taps "Delete", then confirms, Then: the route is removed from their saved routes and cannot be retrieved.	The route no longer appears in the saved routes list. The remaining saved routes are unaffected.	Low

GUI Prototypes

Table 14 - First GUI prototypes for the various windows of the application



Figure 7: Route Beer age verification page

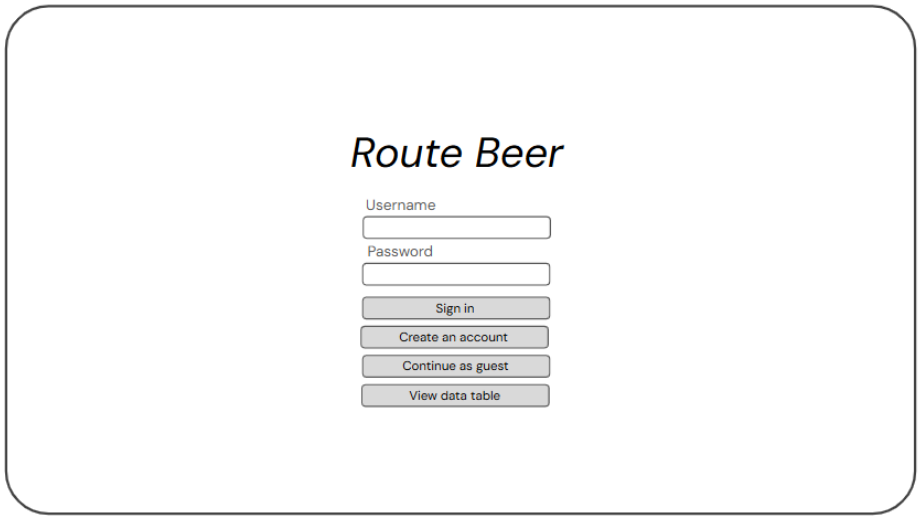


Figure 8: Route Beer login page

Figure 7 and Figure 8 are the prototypes for the login window flow. The idea is that the first window will launch asking the user if they are over 18, providing a yes and a no button. When wondering if “Are you over 18” was too vague of a statement (as it may confuse someone who is exactly 18 years old), it was determined to be reasonable. Upon checking NZ liquor store websites such as Liquor Land and Super Liquor, the term “Are you over 18” is consistently used, so this is statement is sufficient.

If the user clicks no, then the app will run a pop up saying, “You must be over 18 to use Route Beer”, with an OK button, which when pressed will close the app. This follows QR_8, legality, as we are redirecting underage users from our application, which is centred around alcohol. Further precautions to prevent underage users will not be necessary, as age verification takes place at the entrance of all bars. If the user clicks yes, the window will show the username and password fields, with the sign in (UC_8) and continue as a guest option. This GUI design has consistent and visually pleasing aesthetics (QR_6), and usability (QR_3) is supported by well sized and readable text, alongside simple and intuitive navigation.

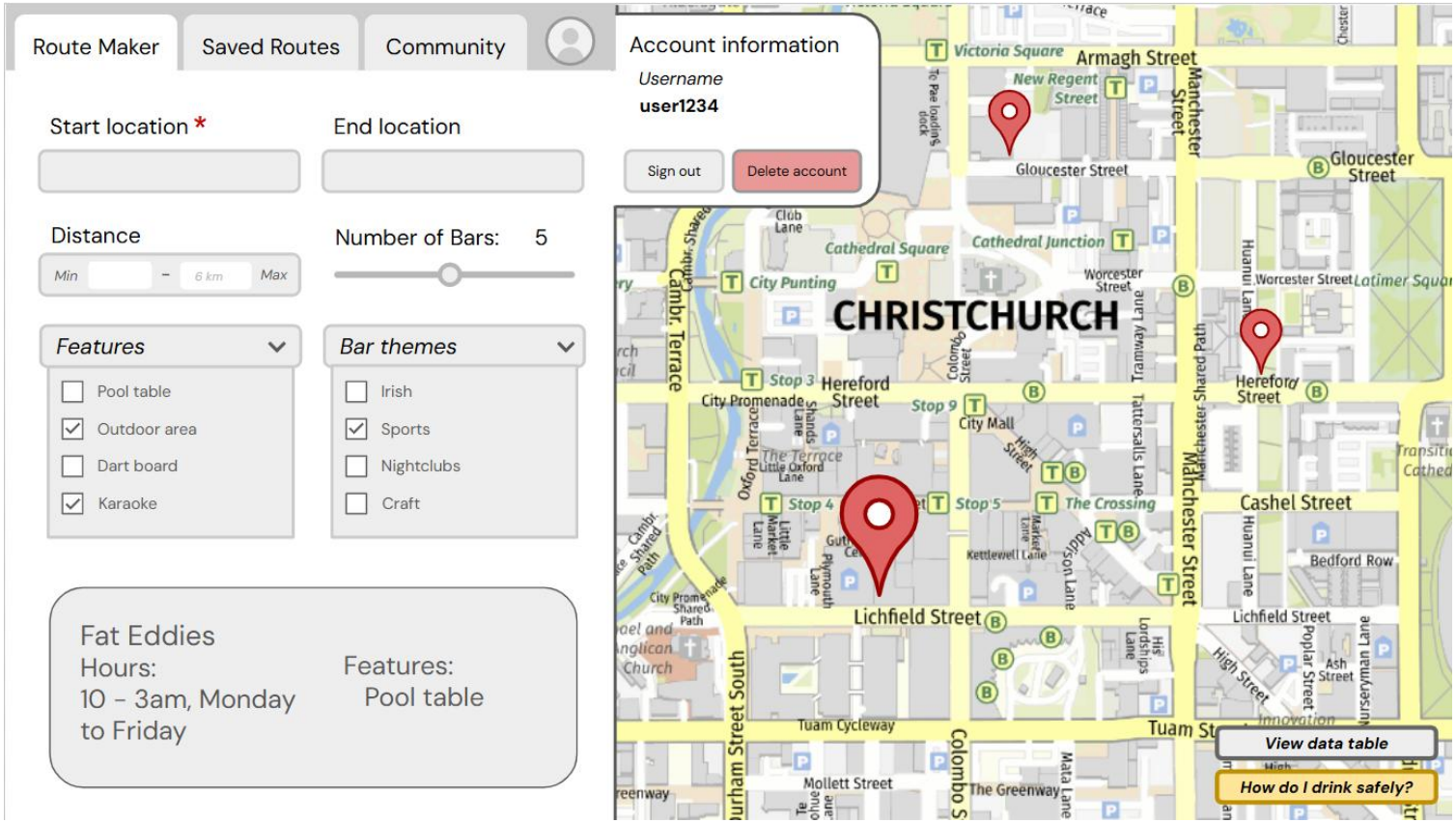


Figure 9: Route Maker tab with a bar selected

Figure 9 is a prototype of the main window, with the Route Maker tab selected. The screen will be split roughly in half, with one side displaying a tab pane and one side the map. On the left are all the parameters that you can select to generate a route (UC_1, UC_2) with the compulsory one having an asterisk. Users set a start/end location, a distance range, number of bars, and filter by Features and Bar themes. This design supports performance (QR_1), as utilising a tab pane allows users to navigate the app without the wait time required when loading and unloading individual windows. Robustness (QR_2) will be enforced on this page, as each field will gain a red border if invalid inputs are given, preventing unexpected data reaching the route generation algorithm, and labels will be shown detailing the cause of any errors. Robustness is also improved by our field choices, using a slider will only allow a valid number of bars, and the distance field

will only allow numeric characters to be entered. All text is reasonably sized and easy to read, and the use of text fields, sliders and drop down checkboxes support usability (QR_3). Using familiar, conventional interface elements reduces the learning curve, which supports our Usability requirement by letting users navigate and complete tasks without guidance. Aesthetics (QR_6) is also followed, as design choices are consistent and are visually nice.

Clicking a pin (e.g. Fat Eddies) shows its hours and features. This screen supports UC_1 (generate with default parameters), UC_2 (generate with specific parameters), and UC_13 (interact with bar locations on map). This serves SH_1 (non-abstinent end users), and SH_3 (abstinent end-users).
The account popup (username, sign out, delete account) supports UC_7 (manage current account), also for SH_1 and SH_3. The “How do I drink safely?” button supports UC_12, as it will display information on alcohol safety when clicked, serving SH_4 (Christchurch City Council), SH_6 (alcohol regulatory/support boards), and SH_7 (Ministry of Justice).

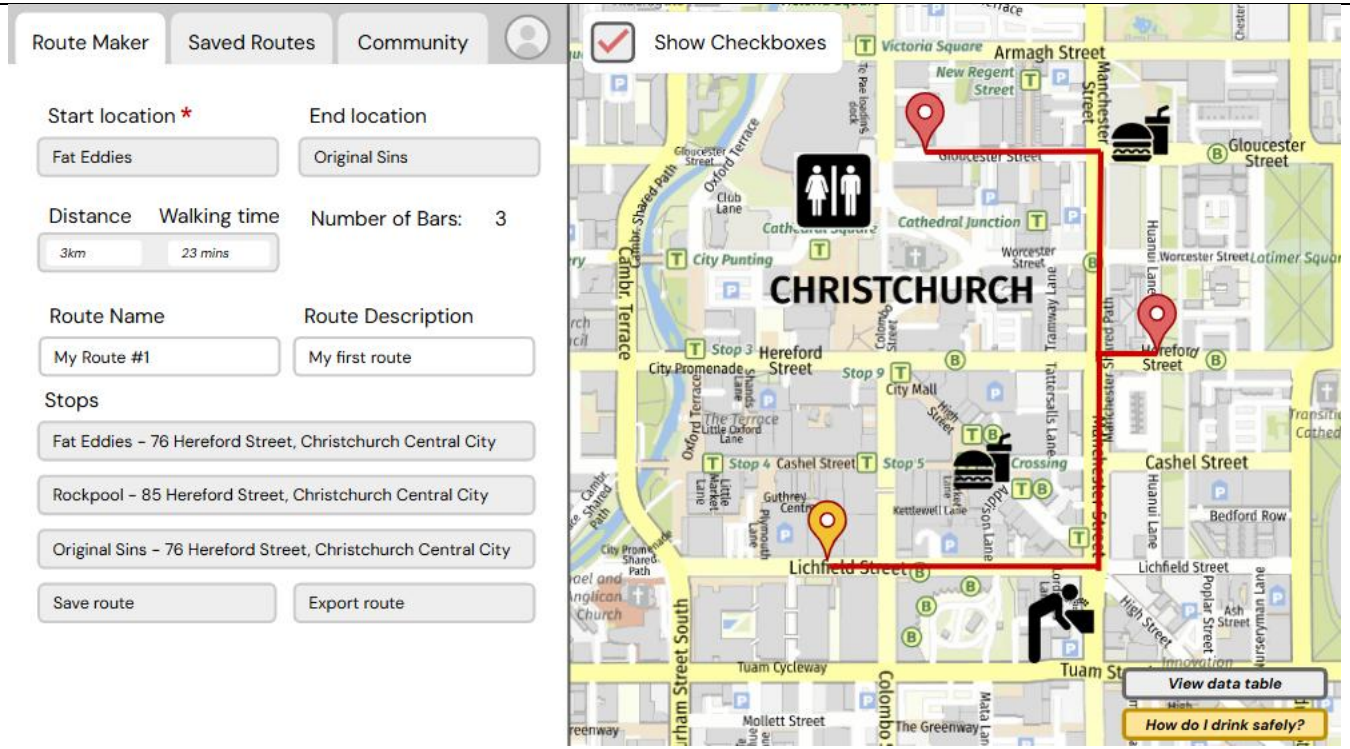


Figure 10: Route Maker tab after route has been generated

Figure 10 shows a generated route: an ordered stop list, walking time, and the path drawn on the map. When a generated route is loaded to the map, a checkbox to toggle amenities such as public toilets, fast food places and drinking fountains that are near the route (UC_14). This is the output of UC_1 and UC_2 and is the point at which UC_5 (save a created route) would occur, again for SH_1 and SH_3. After a route is generated, the start bar will be a different colour to indicate it is the beginning of the route. All the stops in the route are shown in order on the tab pane. This design continues to support usability (QR_3), as the route details are clearly shown, and the route appears instantly on the user's screen with no need for further navigation. The simple organisation and presentation of the route information further enforces this requirement.

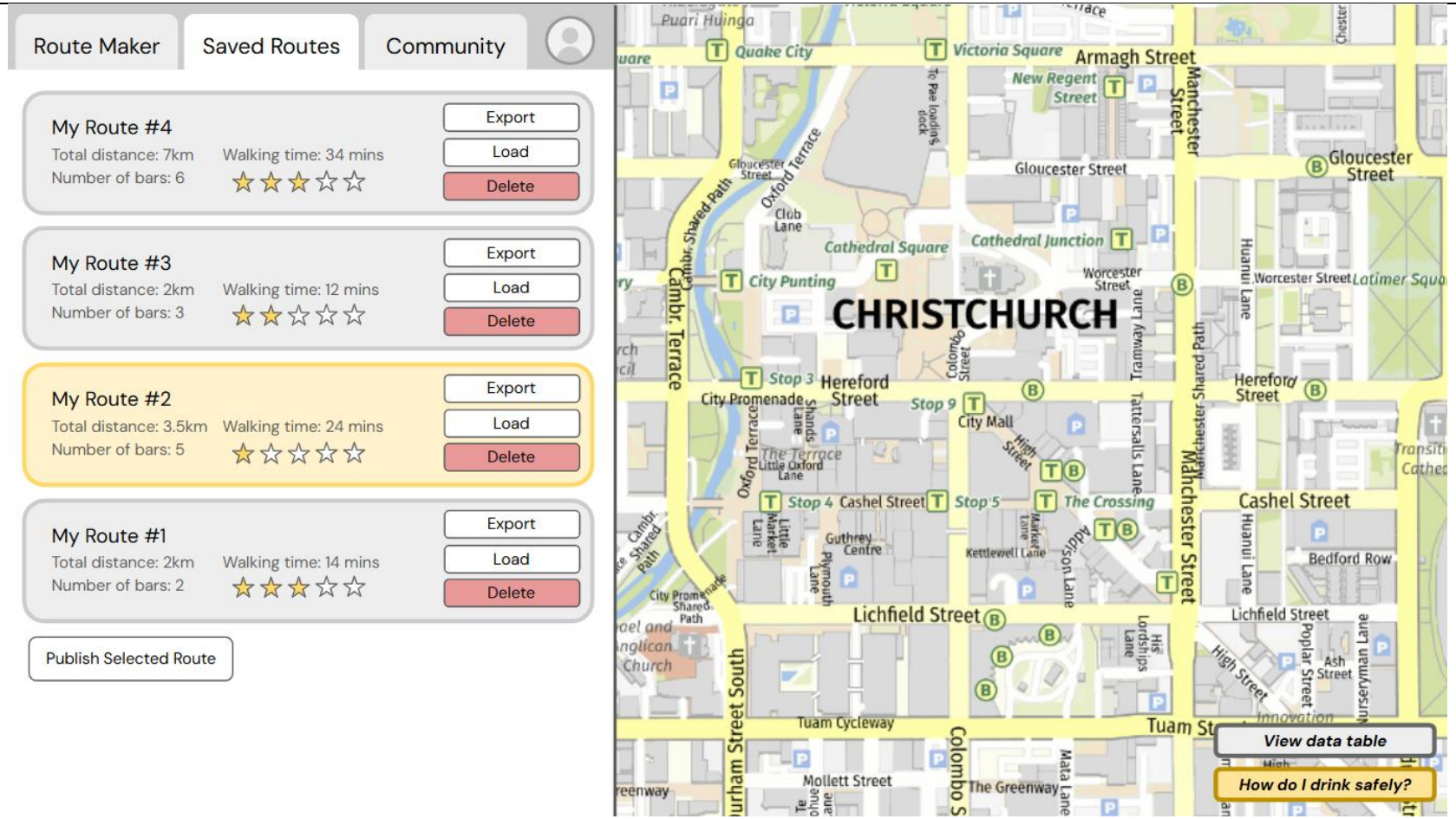


Figure 11: Saved Routes tab on the main screen with a route selected

Figure 11 is a view of the saved route pane. If the user is signed in, the page displays their routes, each with distance, walking time, bar count, and a personal star rating, plus Load/Delete actions. This screen is UC_6 (manage saved routes), and the star rating shown here is the personal rating from a member's own saved route, not a community rating. This serves SH_1 and SH_3, and will be consistent with QR_1 (saved routes should load in under 2 seconds). There is the option to export, UC_3, delete or load any one of the routes. There is also a button to publish a route, which allows a member to publish a selected route to the community page. If the user is not logged in, this page will show a message prompting them to sign in with a button redirecting them to the Setup window. This page follows the same

design as the route maker page, and alongside the effective representation of each induvial route, fits the usability (QR_3) and aesthetics (QR_8) quality requirements.

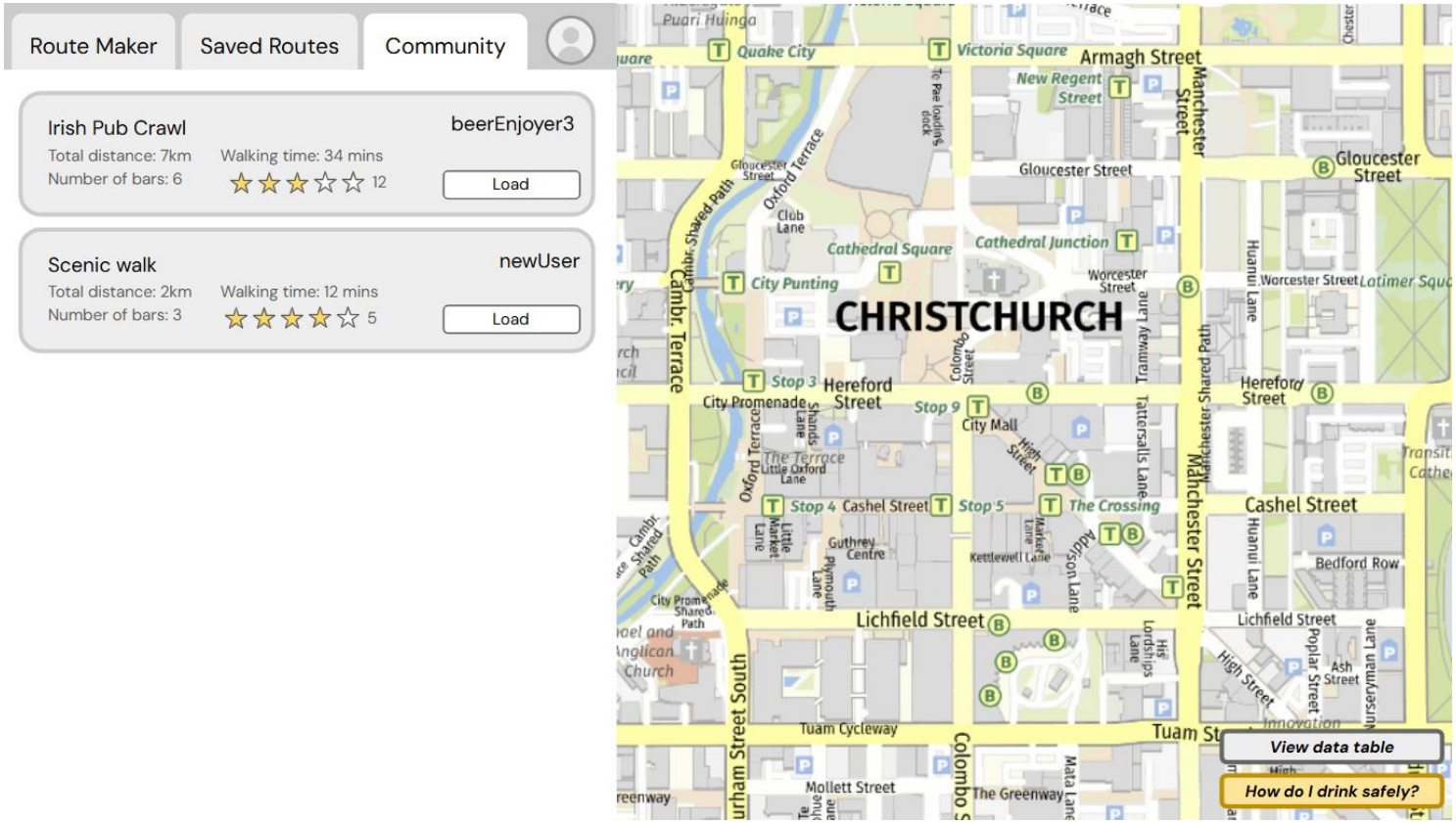


Figure 12: The Community tab showing routes which other users have published

Figure 12 shows the community page. This holds published routes (Irish Pub Crawl, Scenic walk) with a star rating and count, load button, and the username of the member who published this route. This screen allows the user to browse and load community routes (UC_9), and members can rate them (UC_11). The design is consistent with the “Saved Routes” page, continuing the consideration of QR_3 and QR_8. Across all pages, the user will be able to navigate quickly and the text size and font will be large, further satisfying QR_3. The shared design between route cards will also help with maintainability (QR_4), as any edits will reflected in both cards, instead of having to individually update each type.

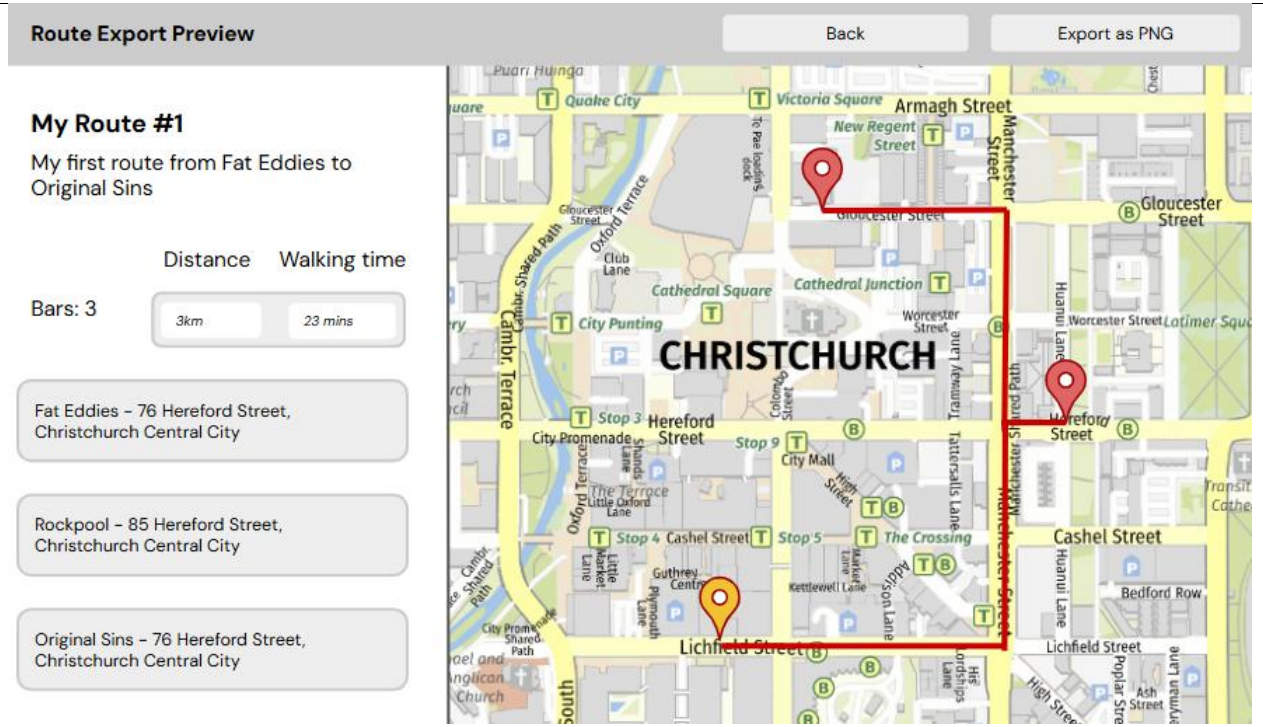


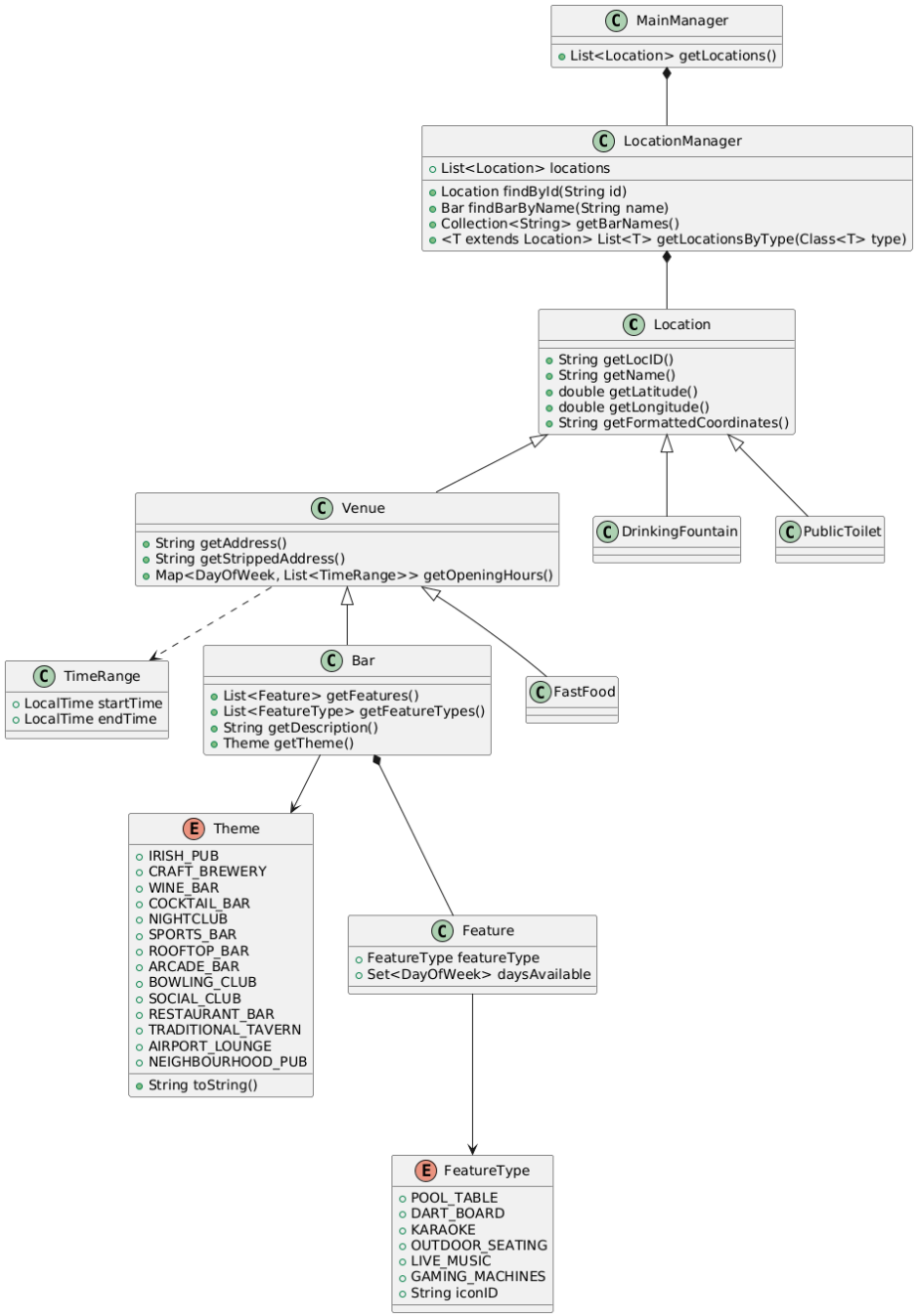
Figure 13: The Export Window preview showing the map, route and details

Figure 13 shows the page fulfilling UC_3, exporting a route. This window will be launched when the export button on a route card is pressed. This window will display all necessary details in a compact layout, and a snapshot of everything below the Route Export Preview header will be saved as a PNG to a location of the user's choice when the Export PNG button is pressed. Design choices are consistent with the rest of the app, and text is clear and readable, fitting QR_3 and QR_8, and taking a snapshot of the current map supports performance (QR_1), as the export screen will load significantly faster compared to re-rendering the map and re-drawing the route and locations.

Detailed UML Class Diagram

(1-2 pages)

Figure A shows how all models for a location are accessed



the base class for all location entities, storing location ID, name, latitude, longitude, and formatted coordinates. DrinkingFountain, PublicToilet, and Venue extend Location directly, while Bar and FastFood extend Venue.

LocationManager manages all domain entities within a single list of locations, providing lookup helpers and type-filtered querying such as getLocationsByType.

Venue is an subclass of Location that adds address properties and an openingHours mapping represented as a Map of DayOfWeek to a List of TimeRange objects, where each TimeRange explicitly encapsulates a startTime and endTime. FastFood extends Venue with no additional attributes. Bar extends Venue and contains a List of Features, a Theme enum, and a description. Theme is an enum representing the establishment's theme. Feature pairs a FeatureType enum with a Set of DayOfWeek indicating the days the feature is available.

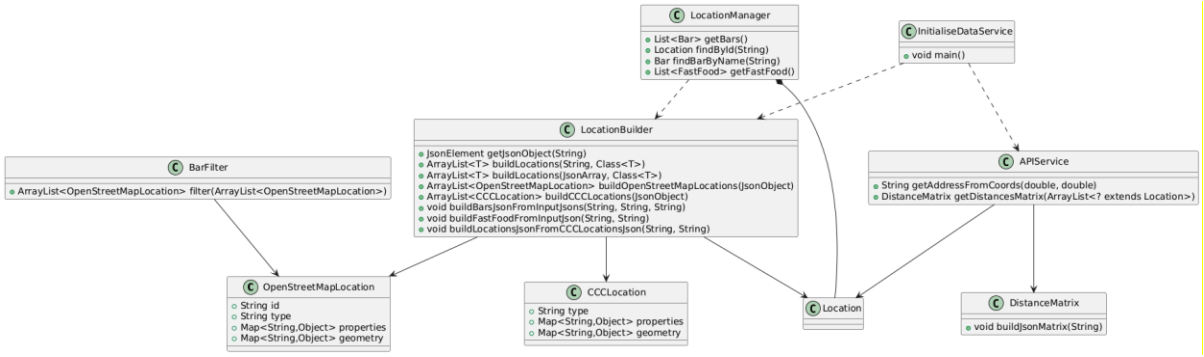


Figure B stores the base Location objects and their subclasses in JSON files; LocationBuilder is used to create these. This observes QR_8, storage optimisation, by minimising calls to the Geoapify API. InitialiseDataService calls buildLocations() to construct the specific location types. In the refactored hierarchy, Bar and FastFood inherit from a Venue class (which adds address details and opening hours managed by TimeRange), while PublicToilet and DrinkingFountain inherit directly from Location. These processes use helper classes: CCCLocation for PublicToilet and DrinkingFountain classes, and OpenStreetMapLocation for Venue subclasses (FastFood and Bar) to help pull the correct data from geojson files. BarFilter is

used to filter bars that may be stored in OpenStreetMap as restaurants, and Bar objects are further enriched with a Theme and a list of Features (using FeatureType to track specific amenities and the days they are available). DistanceMatrix stores the distance from every Bar in the application to every other. This is stored in a JSON file as well, satisfying QR_8, and an efficient distance matrix will improve route generation performance, satisfying QR_1 as well. Finally, MainManager contains a LocationManager which, when the application begins, stores all locations and provides structured access to them (such as fetching locations by type or finding a specific Bar by name). MainManager can be passed to any class in the application that requires it.

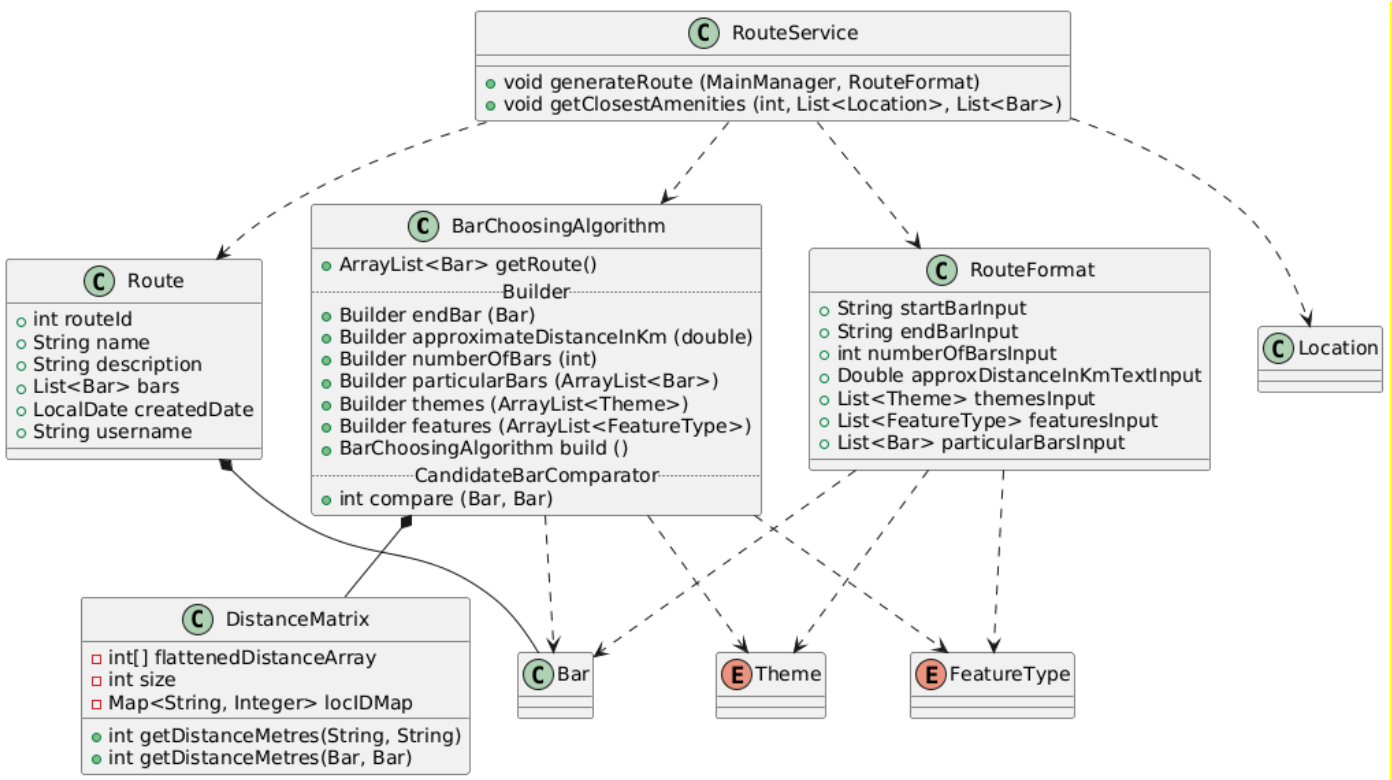


Figure C shows how the BarChoosingAlgorithm (BCA) decides which bars are visited on a Route and is called through RouteService through generateRoute(). Construction uses the Builder pattern, which is nested inside the BCA and shown in

the diagram as a section within the class. The compulsory parameter is startBar. The optional parameters are set through the fluent methods endBar, approximateDistanceInKm, numberOfBars, particularBars, themes and features. Each of these returns the Builder, and build() finally returns the configured BCA. The themes and features parameters are typed by the Theme and FeatureType enumerations, and particularBars is a list of Bar, which is why the BCA has dependency relationships to all three.

The BCA selects bars using a nested CandidateBarComparator, which ranks candidate bars through its compare(Bar, Bar) method, together with a filter that removes candidates failing the user's constraints. Both are applied at each step of the recursion. Distances between bars come from the DistanceMatrix, which the BCA owns through a composition relationship. The DistanceMatrix is loaded from JSON and stores all pairwise distances in a flattened integer array (flattenedDistanceArray) of known size. A locIDMap maps location identifiers to positions in that array. Distances are looked up in metres through getDistanceMetres, which accepts either two location ID strings or two Bar objects. Because distances are precomputed and read directly from the array, they never need to be recalculated during the recursion, which greatly improves the performance of the BCA (QR_1).

RouteService acts as the entry point with generateRoute(MainManager, RouteFormat) taking the user's input, held in a RouteFormat object, and uses it to configure and build a BCA. The RouteFormat carries the raw inputs from the interface: RouteService runs the BCA, turns the chosen bars into a Route, and sets the currentRoute in the MainManager to that Route. RouteService also provides getClosestAmenities(), which finds the nearest amenities for a given set of bars and locations.

Together, these classes implement the functionality of UC_1, where a route is generated from a start bar, and UC_2, which adds the further parameters (end bar, distance, number of bars, particular bars, themes and features) that the Builder supports.

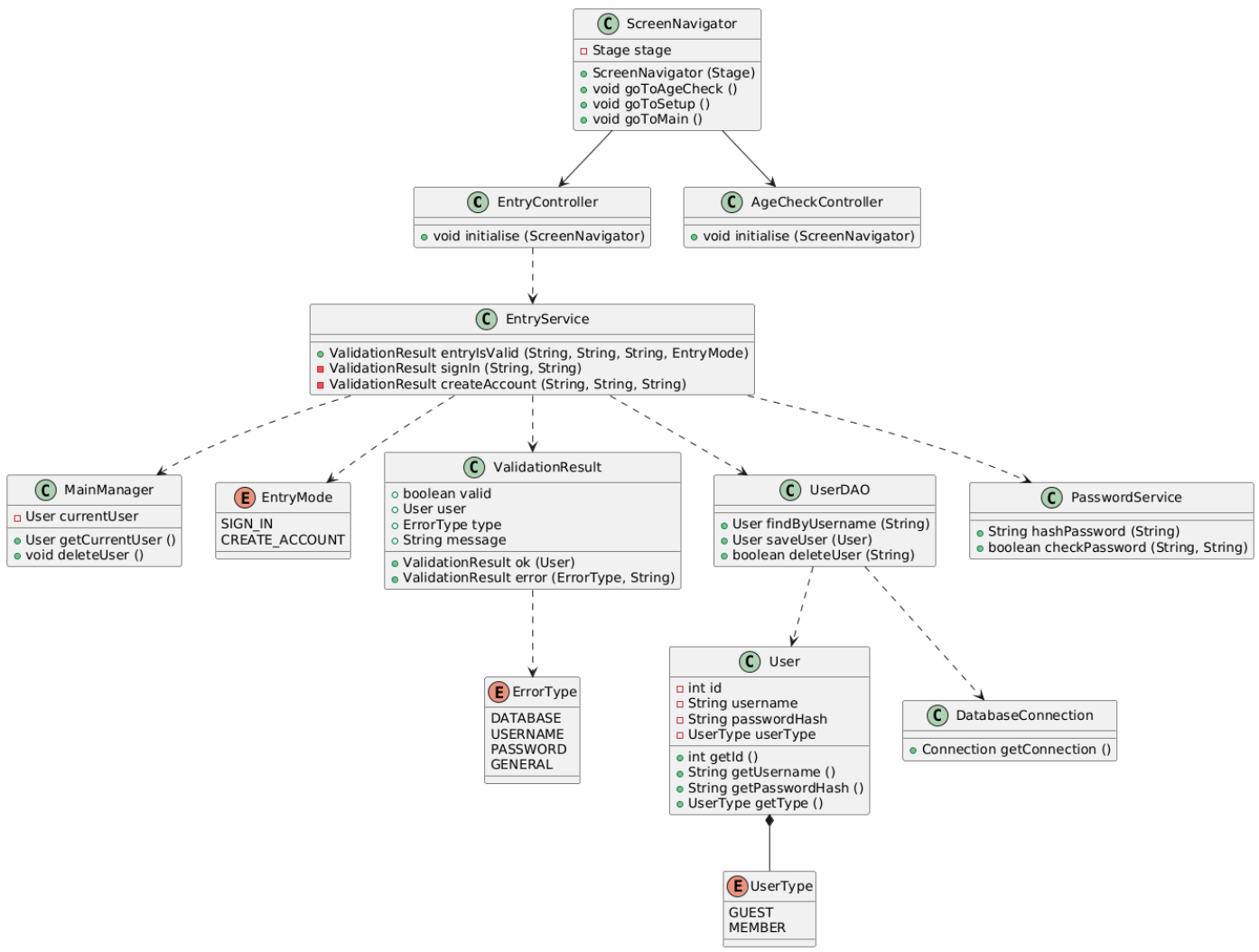


Figure D shows the controllers and services that handle the login functionality. To sign in, the user first loads into ScreenNavigator, which directs them to AgeCheckController to confirm their age. After confirmation, ScreenNavigator directs them to EntryController, which provides the create account and sign in subpages. EntryController collects the username and

password and passes them to EntryService, along with an EntryMode (SIGN_IN or CREATE_ACCOUNT) that determines whether EntryService performs its signIn or createAccount logic.

EntryService carries out the validation and uses UserDAO to look up and persist users. UserDAO provides findByUsername, saveUser and deleteUser, and obtains its connection to the MariaDB database from DatabaseConnection. When creating an account, EntryService first uses UserDAO to check that the username is not already taken. It then passes the password to PasswordService, which uses the BCrypt library to hash it before the new User is saved, ensuring QR_7, account security. When signing in, PasswordService.checkPassword compares the entered password against the stored hash.

EntryService returns a ValidationResult to EntryController. On success, the result is created with ok(User) and carries the authenticated User. On failure, it is created with error(ErrorType, String), where the ErrorType (DATABASE, USERNAME, PASSWORD or GENERAL) and message tell the controller which problem to report. On a successful sign in, EntryService also updates MainManager with the current user, which the rest of the application retrieves through getCurrentUser. A user's UserType is either GUEST or MEMBER, and this determines the access they have to features such as saved routes. MainManager also exposes deleteUser, which removes an account through UserDAO. UC_3 and UC_8 are satisfied by the classes shown here.

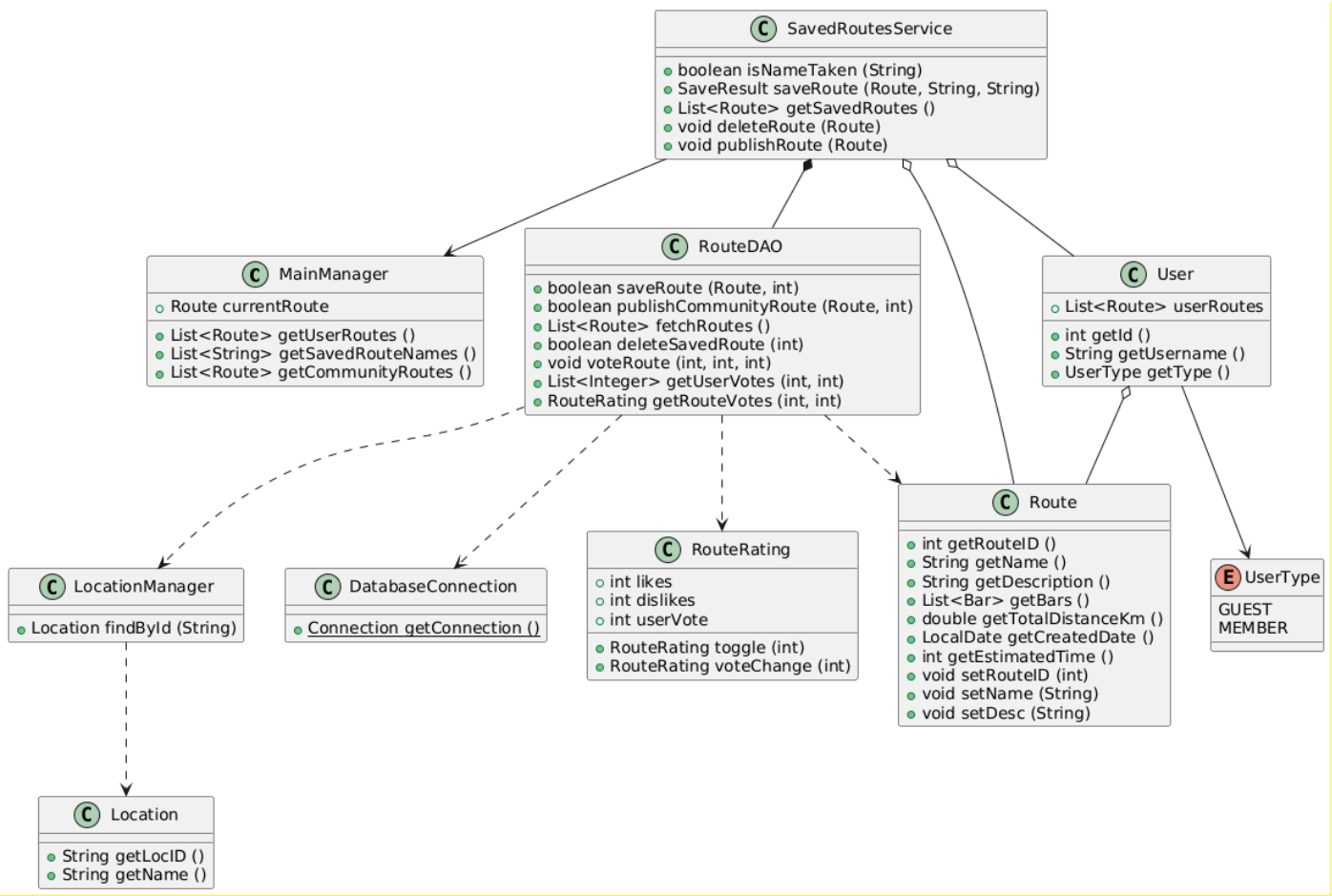
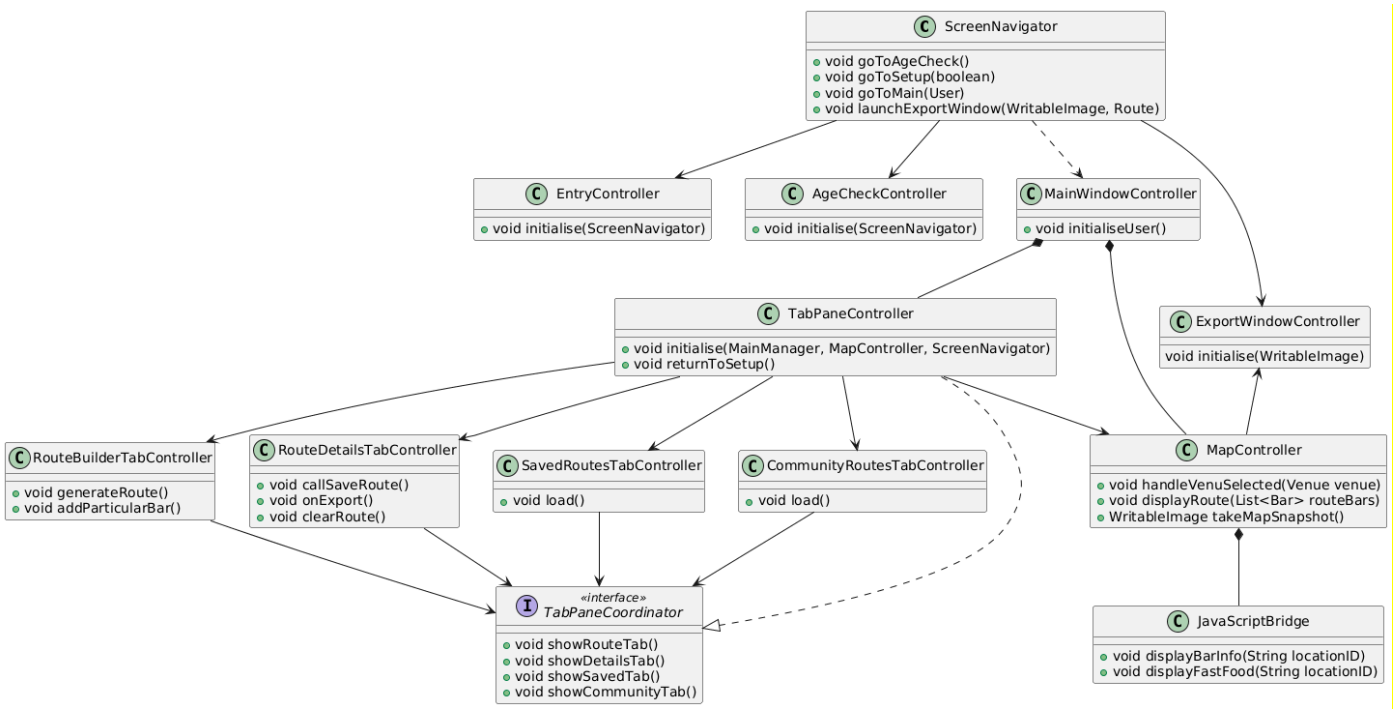


Figure E shows SavedRoutesService, which takes care of route persistence throughout the application. It reads and updates shared state through MainManager and handles User-Route interactions allowing the user to interact with RouteDAO. This satisfies UC_*5*, *UC*_6, UC_9, UC_10 and UC_11.

RouteDAO uses LocationManager to turn stored location IDs back into Location objects (findById) when rebuilding routes from stored data. RouteRating is a small value class holding the number of likes, the number of dislikes and the current user's

vote. Its toggle and voteChange methods return an updated RouteRating rather than modifying the original, so vote state can be recalculated safely. RouteDAO returns it when a route's votes are requested.

Route exposes getters for its ID, name, description, bars, total distance, created date and estimated time, along with setters for the ID, name and description, which are the fields filled in during the save process. Location is a simple class with an ID and a name.



A majority of use cases are satisfied with the controller classes in Figure 19. ScreenNavigator operates what window the application is in, swapping between AgeCheckController, EntryController and MainWindowController. This layout satisfies QR_7 and QR_3, as the application's navigation does not directly interact with data classes and can reach any point of the app from any other point in 3 clicks.

TabPaneController (TPC) implements the TabPaneCoordinator interface, which exposes showRouteTab(), showDetailsTab(), showSavedTab() and showCommunityTab(). Each tab controller (RouteBuilderTabController, RouteDetailsTabController, SavedRoutesTabController and CommunityRoutesTabController) depends only on this interface rather than on TPC itself, so a tab can request a switch to another tab without knowing how the tab pane is built or about any other tab.

MapController uses WebEngine.execute() to run JavaScript within the leaflet map, handling the core use cases (UC_*3, UC*_13, UC_14 and UC_3) function with displayRoute() drawing the bars of a Route on the map, handleVenuSelected() responding to a chosen Venue, and takeMapSnapshot() capturing the map as a WritableImage. Route data for these controllers all use the Route located in MainWindowController, which composes both MapController and TPC. ExportWindowController allows the user to export their Route to a png file, satisfying UC6. It is opened by ScreenNavigator.launchExportWindow(), which receives the map snapshot and the Route, and is triggered from RouteDetailsTabController.onExport(). JavaScriptBridge is owned by MapController, and helps satisfy UC_14 displaying bar information with displayBarInfo().

Testing

Testing process

Iterative development through feature-based development meant that testing was integrated as we went. We used unit, end-to-end, acceptance, and manual testing to ensure that the application runs as expected. Unit testing checks each class and its methods on a low level, as isolated as possible. We tested all our non-GUI classes with the JUnit unit testing package, which covers the core logic of the application. JUnit allows us to write isolated tests for particular methods, allowing assertions that confirm each method does what it should. End-to-end testing checks that the user-experience is what it should be. We used the TestFX package to test JavaFX functionality which utilises a robot to simulate user interactions such as clicking and writing. By simulating how an end user would interact with our application, we can test for feedback of errors to the user, and app flows that cannot be tested with JUnit alone. Acceptance testing (AT) is a higher-level testing checked that the application on a whole does what it was intended to do. We used the Cucumber package for automated acceptance testing; each AT was written as a Gherkin scenario and tagged with its test ID (e.g. @AT_5), so every scenario traces back to a use case. This supports QR_4, maintainability. Automated testing means if anything goes wrong in the future our tests should

immediately pick up on it. The most thoroughly tested component is the route generation pipeline, since it is the backbone of the product. The branch coverage and instruction coverage of non-controller classes were both required to be a combined average of at least 80% each before merging into main. After learning TestFX, the 80% branch and instruction coverage rule applied to all classes including GUI controllers. Writing unit tests was allocated to the person who wrote the functionality within the class. We wrote acceptance tests using Cucumber aligned with our documented acceptance tests to ensure the required use cases were met in the application. We also included manual testing to cover high-level end to end use cases with hard to test features using Cucumber or TestFX.

Discussion of Test Results

Our current product version has 217 unit tests and 16 acceptance tests. The unit tests cover almost all of our models, services, and exceptions.

route-beer

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
seng202.team5.gui.control_panel		72%		48%	102	204	173	610	27	105	1	8
seng202.team5.gui		53%		45%	42	74	123	259	26	53	2	6
seng202.team5.services		83%		71%	28	89	40	208	17	61	0	5
seng202.team5.services.route_generation		96%		90%	22	193	16	508	3	84	0	6
seng202.team5.models		85%		63%	13	58	10	92	2	36	0	6
seng202.team5.services.one_off_scripts		80%		90%	5	17	17	81	3	6	1	2
seng202.team5.models.locations		93%		86%	13	66	9	137	6	41	2	9
seng202.team5.gui.entry		95%		93%	4	31	7	89	3	22	0	2
seng202.team5		0%		n/a	3	3	5	5	3	3	1	1
seng202.team5.exceptions		85%		n/a	2	11	5	27	2	11	0	7
seng202.team5.data		98%		89%	7	50	2	198	1	22	0	4
seng202.team5.services.entry		98%		100%	1	25	1	42	1	11	0	3
seng202.team5.models.types		100%		n/a	0	10	0	43	0	10	0	5
Total	1,635 of 10,268	84%	189 of 718	73%	242	831	408	2,299	94	465	7	64

Figure 14: Overall JaCoCo

Our overall code coverage is at 84% instruction coverage and 73% branch coverage, determined by JaCoCo, Figure 14. This hits our overall average requirement of 80%.

route-beer

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
seng202.team5.gui.control_panel		72%		48%	102	204	173	610	27	105	1	8
seng202.team5.gui		53%		45%	42	74	123	259	26	53	2	6
seng202.team5.gui.entry		95%		93%	4	31	7	89	3	22	0	2
Total	1,192 of 3,986	70%	123 of 251	50%	148	309	303	958	56	180	3	16

Figure 15: GUI JaCoCo

Our GUI coverage from controller classes was what brought our coverage down, having 70% and 50%, instruction and branch coverages respectively, Figure 15. Alongside QR_4 maintainability, this is important for QR_2 robustness. We check in our GUI JUnit tests for errors being thrown to user; the program is robust when they can see what is going wrong, especially for route generation failure. Testing controllers required TestFX, which took longer to write and run than standard JUnit tests and was learnt later in development which is why our GUI coverage was lower compared to models, service, data, and exceptions.

route-beer

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
seng202.team5.services		83%		71%	28	89	40	208	17	61	0	5
seng202.team5.services.route_generation		96%		90%	22	193	16	508	3	84	0	6
seng202.team5.models		85%		63%	13	58	10	92	2	36	0	6
seng202.team5.services.one_off_scripts		80%		90%	5	17	17	81	3	6	1	2
seng202.team5.models.locations		93%		86%	13	66	9	137	6	41	2	9
seng202.team5		0%		n/a	3	3	5	5	3	3	1	1
seng202.team5.exceptions		85%		n/a	2	11	5	27	2	11	0	7
seng202.team5.data		98%		89%	7	50	2	198	1	22	0	4
seng202.team5.services.entry		98%		100%	1	25	1	42	1	11	0	3
seng202.team5.models.types		100%		n/a	0	10	0	43	0	10	0	5
Total	443 of 6,282	92%	66 of 467	85%	94	522	105	1,341	38	285	4	48

Figure 16: Non-GUI JaCoCo

Our non-GUI classes had high coverage at 92% instruction and 85% branch coverage respectively, Figure 16. This also helps with QR_2 robustness as it is checking inputs from the user for methods; incorrect inputs should be caught and handled appropriately. The JavaScriptBridge and MapService classes had lower coverage than others because of their difficulty to test, requiring interactions with other languages alongside JUnit. BarChoosingAlgorithm also had lower coverage because of the large number of branches on parameter inputs for generating a route.

High JaCoCo coverage in non-GUI packages is what gives us confidence that the isolated application methods will function as intended. Instruction coverage refers to bytecode, different tasks the application will run, whereas branches are each different task an if or switch can cover. High branch and instruction coverage don't guarantee that our application solves the problem it was made to, just that each method we have does what we assert in to in JUnit. We went beyond blue-sky scenarios, testing alternative and exceptional flows, to ensure that we hit many branches and instructions.

We automated 16 of our 22 acceptance tests in Cucumber, covering 8 of the 9 highest priority ATs. These covered creating account, route generation, managing saved routes, saving routes, managing account, and signing in. Cucumber is fast and repeatable for logic, but we found that testing the controllers or display functionality are best using other high level testing methods. The remaining acceptance tests and use cases are covered with end-to-end testing using TestFX and manual testing. Together, we are confident that all the 22 ATs are covered. The weakness is that the manual tests must be repeated by hand for each release

The stakeholders covered by these ATs and end-to-end tests are SH_1, SH_2, SH_4, SH_6, SH_7. These are the end users of the app and the stakeholders who are concerned with the compliance with the Sale and Supply of Alcohol Act (2012). These stakeholders are the ones who have primary input / impact on the app. Having 8 of our top 9 priority ATs covered, means that the core functionality of the application is tested and robust, backing QR_2. The safety check at startup (UC_12) caters to SH_4, 6, 7, ensures that our application should be used responsibly and only by adults.

We used some manual testing in the application to test areas that other methods were unable to reach. This included every acceptance test and covered all our medium and high-priority use cases. It was done by writing out a set of simple instructions for each test which the tester can follow and observe the outcome. Of the 24 tests, 22 passed. Within the two failed tests, one failed because the user feedback messaging regarding a failed route generation was meaningful and helpful,

but not the same as the expected outcome of the test. The other test failed because although a guest cannot physically save a route, there is no user feedback to communicate this to them. The reason we did manual testing was to cover the cases where JavaFX would have to be used when doing Cucumber testing. These missed Cucumber tests are then covered by both end-to-end testing using TestFX and manual testing. Through TestFX and manual testing, we managed to cover all the acceptance tests that could not be covered using Cucumber.

Current Product Version

The project is a Java application that plans pub crawl routes across Christchurch venues. In deliverable 3 (D3) we built on our minimum viable product by adding features which allow more customisation of routes and interaction with published, community routes. Our core app feature is the route custom route generation. This is where a user can select a start bar along with additional parameters (end bar, number of bars, approximate distance, bar features, bar themes, particular bars) which will be used to create a walkable route between an optimized sequence of bars. This relates to UC_1 and UC_2 which are the fundamental functionalities of the application. End users can create an account (UC_4) or use the application as a guest. If the user is signed into a registered account (UC_8), a generated route can then be saved to the university database for persistent storage (UC_5). It can be loaded from a logged in user's "Saved Routes" tab (UC_6). When a user has a loaded route on the map, it can be exported as a PNG to a file of the user's choice (UC_3). There is a community tab in which registered users can publish saved routes and all users can load published routes (UC_9, UC_10). In this community tab, a member user can like or dislike published routes (UC_11). There is safe alcohol consumption messaging throughout the app with an age check on entry and a safe drinking footer permanently when using the app (UC_12). When browsing the map, user can select bar locations and can view a popup of the information about the bar (name, address, opening hours, theme, features) (UC_13). When the user has a route displayed on the map, there are toggleable amenities (fast food, toilets, drinking fountains) which are fit along the route (UC_14). The fast-food locations can be viewed similarly to bars when clicked.

Overall, our core vision remained consistent throughout the project, just with certain features changing priority from what we had planned. During this deliverable, the key features that we implemented were the rating of community routes, and being able to select bars the user would want in the route. The community ratings are in a likes/dislikes format where a user can either like or dislike a route and that choice will be saved unless changed or their account is deleted. The introduction of the

being able to select particulars bars caused friction with the route generation algorithm, causing it to become less efficient. It was very important for us to fix this because our QR_1 is performance. We did this by filtering the particular bars before passing the final list to BarChoosingAlgorithm instead of filtering at the end. This improved performance significantly. The overall quality of our app has been improved as well with the UX/UI polished as well as increased user feedback for errors. Another feature we implemented was the numbering of bar pins along a generated route. This allows the user to have more clarity on the sequence of their route. We did this by putting the cardinality of the bar on top of the pin as opposed

Some requirements from our original product description remain unimplemented after D3. One of these was being able to browse the map and select the start location from the pin themselves. This would allow the user more autonomy and understanding about where their route will start. It was decided in the end to prioritise other features such as community routes because user can still see the names of pins and manually input then into the start bar input. Another feature would be route editing, where a user could edit a generated by removing certain bars or changing the order of them. However, with the scope of project, and how our team's opinions changed over the course of the deliverable, we decided that these features were not as critical to the implementation of the app as the ones we did implement.

A drawback that we found from developing our application was the availability of all the data we need. From the sources we found, some bars and fast-food locations were missing key attributes such as opening hours. To combat this, we used and LLM to create faux data to supplement the dataset. This allows us to have a fully functioning application where all locations have at the minimum; a name, opening hours, address, theme, and a description as well as many bars having features.

We are most proud of the route generation feature of the product. This is the backbone of what separates our application from others in the market. Its customizability allows the users more freedom than our competitors. The semi-greedy algorithm aims to space bars evenly between the start and the finish based on the approximate distance given by the user. We created a distance matrix early in the project with the walking distance between all bars in the database with each other. The distances are stored in distance-matrix.json and are used by the BarChoosingAlgorithm class which holds the core logic for route generation. This has allowed us to generate hundreds if not thousands of different routes in under a second. With this implementation, we can be confident we are choosing a high-quality route from the user's given parameters.

If we are to carry this project on in the future, there are several features we would look to implement for the betterment of the app. One feature would be to allow the user to pick the start, end, or particular bars while browsing the map. They will be able to select a pin on the map and have an option to “Set as start location” for example. Another feature we may look to implement is giving the user multiple routes at a time and letting them choose which one they the most. This will give the user more control over their selected route. Another potential feature is to introduce more analytics and show data about the most popular bars at particular times. This could be relayed onto bar owners which can cater to this increased traffic.`

Deployment Model

Figure 13 is a diagram illustrating the deployment of the application. The RouteBeer.jar is run on the lab machine through the Java virtual machine. This artifact stores a collection of JSON files containing location information which is built into Java objects at runtime, then stored as a list in a local dataset. From these bars, a distance matrix was constructed with the API GeoApify. This distance matrix stores the distance from each individual bar to every other bar in a JSON file. This allows for extremely quick route generation, and as storing this data locally will reduce loading times in comparison to pulling data repeatedly from APIs, reinforcing (QR_1), alongside (QR_8), storage optimisation. The map data will be pulled from OpenStreetMaps. and UC’s CSSE route API is used for drawing the route path. Member account details including usernames, hashed passwords and saved routes are stored on University of Canterbury’s Database mariadb, via TCP/IP, supporting

account security (QR_7). This means members can log in from any lab computer. This database will also store published community data, such as public routes and their ratings. The application can output route export images, which will be downloaded to the lab computer.

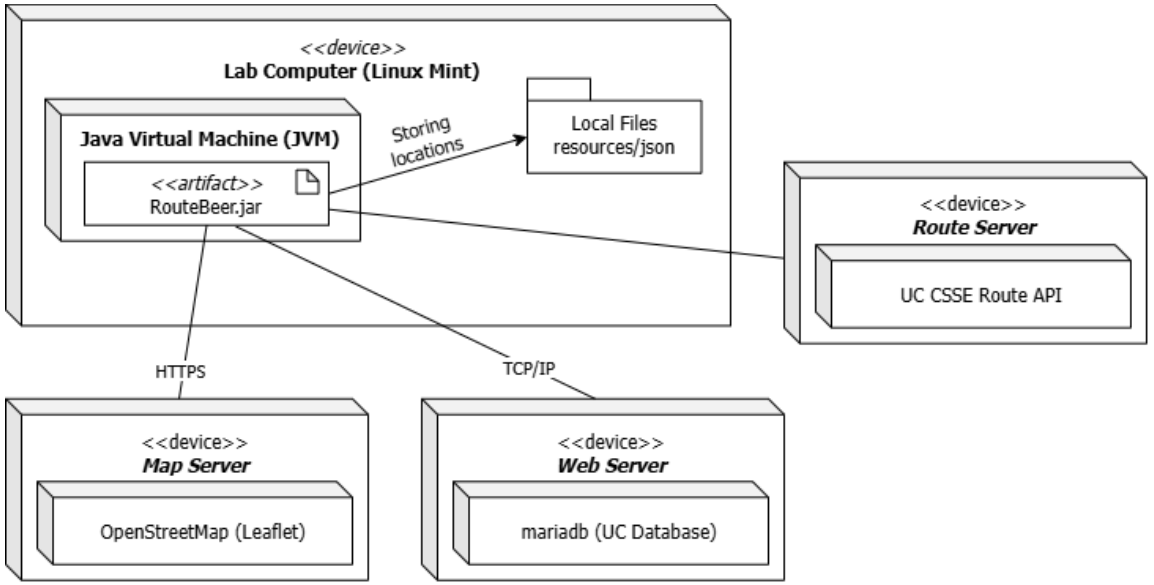


Figure 17 - Deployment Model Diagram for RouteBeer

Risk Assessment

Table 15 shows the evaluation of potential risks relating to the development and deployment of the application. Each risk is measured on two scales ranging from 1-5, 1 being a very unlikely occurrence or negligible impact, and 5 being almost certain to occur or catastrophic consequence. Likelihood measures the probability that the risk will occur during the project and impact measures the severity of the consequence. Exposure is the product of the two scores and lets us measure where to spend time and effort on mitigating risks. A score of 1-6 (Green) is considered acceptable where we should be conscious of the risk, but no major mitigation strategy is necessary. A score of 8-12 (Yellow) is considered medium risk and to treat where applicable. A score of 13-25 (Red) is considered an unacceptable risk and must be treated before continuing the project.

Table 15: Risk Assessment for project

ID	Description	Likelihood	Impact	Exposure	Consequences	Prevention/Mitigation	Realized Exposure
RA_1	A team member becomes unavailable at a crucial time	4	4	16	A part of the project stalls. Knowledge is held by one person. Major time/work pressure on the rest of the team. Especially vital with only having four team members.	Assign a primary and backup owner to each module, recorded on the Trello board, to avoid individual knowledge ownership. Keep documentation as part of the definition of done so work can by picked up without the primary worker. Pair up for working on high risk / important modules which are scheduled into planning. This reduces impact because a persons can continue stalled work.	3x3 = 9
RA_2	Venue data is inaccurate or out of date (e.g. opening hours, address, accessibility).	4	3	12	The users are directed to closed or wrong venues. There is then a loss of trust and unfair effect on pub owners	Source venue data from the City Council open dataset which removes manual transferring errors. Ensure we pull the data at	2x3 = 6

						least once per month which keeps the data up to date.	
RA_3	A third-party routing API is rate-limited, deprecated, or behind a paywall	3	4	12	Route generation, the main feature, breaks. The map may become unavailable	Put routing behind a service interface. Stay inside free token quotas and limit API calls by pulling all bar info at the start and store it in a local json file.	2x3 = 6
RA_4	Optimistic estimates and scope cause features to be neglected or left behind	4	3	12	Incomplete build at the deadline. Requirements unmet. Rushed and low-quality code.	Define MVP of core idea explicitly while also leaving opportunities for scope extension. Timebound tasks and review progress against the deadline each iteration.	3x2 =6
RA_5	The app does not comply with the Sale and Supply of Alcohol Act 2012 (e.g. no age acknowledgement or promotes irresponsible drinking)	2	5	10	Legal and ethical breach. Unjustifiable occurrence	Add an age acknowledgement gate. Align project features with aspects of Act. Include links and information to alcohol help related services.	1x5 = 5
RA_6	Uneven contributions across the team cause conflict, with some team members doing disproportionate work.	3	3	9	Resentment between members and breakdown of the team. Work overload increases capacity for mistakes. Project slows.	Implement a yellow card policy. This punishes poor teamwork for a member. Raise internal conflicts early and keep teammates accountable.	2x2 = 4
RA_7	Location or personal data is collected or stored, breaching user privacy	2	4	8	Breach of privacy act. There is a loss of user trust and potential data misuse.	Collect minimum personal data needed. If anything must be stored, document what and why to align with Privacy Act.	1x3 = 3
RA_8	The app design encourages excessive or rapid consumption of alcohol, conflicting the NFR	2	4	8	Ethical and reputational harm. Non-compliance with NFR 8. Potential real-world harm to users.	Have responsible drinking messages / support section. Default to walking routes.	2x3 = 6
RA_9	Poor upkeep of git repository leads to large	4	2	8	Lost project time. Unstable build. Blocked out teammates.	Pull and commit often which keeps changes small so merge conflicts can be managed effectively. Utilise	2x2 = 4

	merge conflicts and a fragmented main branch					CI / CD pipeline to run tests on every push so bugs are detected as early as possible. Keep up to date documentation so conflicting code can be understood efficiently and resolved effectively.	
RA_10	Local database corrupts or experiences data loss.	2	4	8	Loss of stored data. Time lost rebuilding. Potential unstable build if data and database schema fall out of sync.	Keep database rebuildable from source datasets. Store source data in repo.	1x3 = 3

The risk assessment table spans a variety of both likelihoods and impacts, and the spread is deliberate. Focusing only on high likelihood events such as losing a team member for a while (RA_1) would risk overlooking low likelihood, high impact events such as RA_5, with the reverse also true. The (likelihood x impact) matrix mitigates both risks and allows for a clearer approach.

Before the mitigation strategy, there were standout risks which had to be managed, RA_1 (Red), as well as inaccurate venue data (RA_2), being rate-limited by an API (RA_3), and overestimating our team's performance capacity (RA_4) (Yellow). These four risks drove a lot of our planning decisions because they are the risks which are most likely to cause the project to fail or fall short. However, this does not mean the other risks should be neglected. We will put in the due effort and time relative to each level of risk for the various situations.

After mitigation, RA_1 remains the only risk above an acceptable level. We consider this acceptable rather than failure of planning or mitigation. The chance of a team member becoming sick or unavailable is not something we can remove, so we instead reduce its impact through backup ownership and pairing, with a reduced-scope plan to fall back on.

Every other risk that started in yellow or green ends up in green after mitigation. We take this as validation that our proposed strategies are proportionate rather than surface measures. This is acceptable because there are sufficient plans in place to either reduce both the likelihood and impact or one factor enough to ensure the project can move forward successfully. The plans put in place are realistic and able to be adjusted in the future, shou situations change or new risks emerge.

Project Plan

The major milestones for this project will be the three deliverables; each of these has a hard due date, shown as bold italic in Figure 22. Minor milestones are to guide the completion of the application and are idealised deadlines. Almost all minor milestones are due a few days before the deliverable they are due. This is to allow for sufficient buffer time for scope creep and additional feature aspects not included in the original plan.

Our MVP (Minimum Viable Product) is guided largely by the project outline, but additional package milestones have been determined based on this project. The MVP at each major milestone is as follows; Deliverable 1 must have Feature Package A and the project documentation complete, Deliverable 2 must have Feature Packages B and C, and Highlight Package E complete, and Deliverable 3 must have Highlight Package D.

Deliverable 1's Plan B is to reduce the quantity of work done on the application itself to prioritise the documentation. Feature package A must be completed with Deliverable 1 but doesn't need to be high quality or have additional features. For Deliverable 2, the Plan B is to reduce the scope or drop entirely highlight package E from the due date. This could get offset to Deliverable 3. Plan B for Deliverable 3 is to reduce the scope of highlight package D. This would involve reducing or removing public route ratings, personal route ratings, and public route viewings in that order.

In the optimistic situation that we are ahead of our timeline at any point, we can pull future milestones closer and add additional feature/highlight packages. One idea for D3 is to include highlight package B, implemented by allowing bar owners to view how often routes with their bar are saved. This could be used by them for analytical reasons and could then be included in advertising or promotional events as a commercial option. Another idea is Highlight Package A/Feature Package E where we allow users to comment/report on busyness, quality, or other features of a bar and share this information with other users. These statistics could be displayed in a graph or chart and would be utilised when generating a route or for users choosing if it is satisfactory. A final idea is to add challenges at each bar, like popular pub golf games from influencers (Friends From Work, 2026). These could be themed at each bar or have the overall theme of the entire crawl.

We are aware that this SENG202 project is not the only thing going on in our lives and we will account for this when structuring our project timeline. For example, the COSC265 test is during D3, and our entire team is likely to divert a lot of time to studying for this. We will look to move our effort around by shifting potential work from that week and spreading across the other deliverable weeks.

Appendix

Table 15 - SUS Scores of users of application

I think that I would like to use this application frequently.	I found the application unnecessarily complex.	I thought the application was easy to use.	I think that I would need the support of a technical person to be able to use this application	I found the various functions in this application were well integrated.	I thought there was too much inconsistency in this application.	I would imagine that most people would learn to use this application very quickly.	I found the application very cumbersome to use.	I felt very confident using the application.	I needed to learn a lot of things before I could get going with this application.	
3	2	4	1	5	1	3	2	5	1	**82.5**
5	2	4	1	3	1	5	3	4	3	**77.5**
2	4	3	2	4	1	4	2	5	2	**67.5**
4	2	5	1	5	1	2	1	4	1	**85**
4	3	4	2	4	2	4	1	4	1	**77.5**
5	1	5	2	5	2	5	1	5	2	**92.5**
									Average Score	**80.4**

Table 16 - Task Completion Rate of application

Create Account		Generate Route		Save Route	Export Route	Publish Route
with name and password of their choice		**with parameters "the foundry" to "fat eddies" 5km, visiting 6 bars, a bar with gaming machines, and an Irish pub**		**save the route they've generated**	**save the route as a .png**	**add that route to community tab**
1		**1**		**1**	**1**	**1**
1		**1**		**0.5**	**1**	**0**
0.5		**0.5**		**0.5**	**1**	**1**
1		**1**		**1**	**0.5**	**1**
1		**1**		**1**	**1**	**1**
1		**1**		**0.5**	**1**	**1**
					Average Completion Rate	**87%**
Key: **1 = able to complete** **0.5 = able to complete with difficulty** **0 = couldn't complete without assistance**						

Table 17 - Readability of Application

QR_4: Readability
easy to read and comprehend?
Yes (1)
Yes (1)
Mostly (0.5)
Yes (1)
Yes (1)
Yes (1)

Readability rate
92%